Практическая работа 4

Выполнение задач обработки данных в распределённых вычислениях

Основная серия проводится при AQE=false, shuffle.partitions=8, autobroadcast=-1. Все времена измеряются внутри одной SparkSession в Ubuntu VirtualBox, а не берутся из методички. Самостоятельный вариант B: позиции заказов и продавцы, группировка по seller_state.

ФИО: Албахтин И.В.

Группа: ИНБО-12-23.

Дата выполнения: 26.10.2026.

Полный Run All завершён 26.10.2026 00:02:19 МСК: 12 кодовых ячеек, ошибок 0. Стенд: TIABD-Albakhtin-IV.

Вспомогательные модули и инструкции запуска.

In [1]:

```
from pathlib import Path
import json
from pyspark.sql import SparkSession, functions as F, types as T
from pyspark import StorageLevel
from lab_helpers import environment, records, save_json, plan, measured, ui_data,
schema_signature
from capture_pr4 import checkpoint
ROOT = Path.cwd()
STAGING = ROOT / "data" / "spark_input"
OUT = ROOT / "exports"
OUT.mkdir(exist_ok=True)
def read_csv(name, schema=None, infer=False):
    reader = spark.read.option("header", True).option("encoding", "UTF-
8").option("escape", '"').option("mode", "FAILFAST")
    if schema is not None: reader = reader.schema(schema)
    return reader.option("inferSchema", infer).csv(str(STAGING / f"{name}.csv"))

from time import perf_counter
from statistics import median
spark =
(SparkSession.builder.master("local[*]").appName("TIABD_Practice_4_Albakhtin_IV")
    .config("spark.driver.memory", "1536m").config("spark.sql.shuffle.partitions",
"8")
    .config("spark.sql.adaptive.enabled",
"false").config("spark.sql.autoBroadcastJoinThreshold", "-1")
    .config("spark.ui.bindAddress", "0.0.0.0").getOrCreate())
spark.sparkContext.setLogLevel("WARN")
sc = spark.sparkContext
result =
{"environment":environment(spark),"source":json.loads((ROOT/"source_manifest.json"
).read_text()), "metrics":{}}
result["environment"]["driver_memory"] = sc.getConf().get("spark.driver.memory")
result["environment"]["autobroadcast"] =
```

```
spark.conf.get("spark.sql.autoBroadcastJoinThreshold")
print(json.dumps(result["environment"],ensure_ascii=False,indent=2))

26/10/26 00:01:35 WARN Utils: Your hostname, tiabd-albakhtin-iv resolves to a
loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
26/10/26 00:01:35 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).

26/10/26 00:01:35 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

{
  "python": "3.10.12",
  "java": "openjdk version \"17.0.20.1\" 2026-08-18",
  "spark": "3.5.7",
  "os": "Linux-5.15.0-198-generic-x86_64-with-glibc2.35",
  "cpu": "Architecture:                            x86_64\nCPU op-mode(s):
32-bit, 64-bit\nAddress sizes:                         46 bits physical, 48 bits
virtual\nByte Order:                            Little Endian\nCPU(s):
2\nOn-line CPU(s) list:                     0,1\nVendor ID:
AuthenticAMD\nModel name:                               AMD Ryzen 7 9700X 8-Core
Processor\nCPU family:                           26\nModel:
68\nThread(s) per core:                    1\nCore(s) per socket:
2\nSocket(s):                              1\nStepping:
0\nBogoMIPS:                               7600.00\nFlags:
fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush
mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt rdtscp lm constant_tsc rep_good
nopl nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16
sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm
cmp_legacy cr8_legacy abm sse4a misalignsse 3dnowprefetch ibpb vmmcall fsgsbase
bmi1 avx2 bmi2 invpcid rdseed adx clflushopt sha_ni arat flush_l1d sme\nHypervisor
vendor:                     KVM\nVirtualization type:
full\nL1d cache:                                96 KiB (2 instances)\nL1i cache:
64 KiB (2 instances)\nL2 cache:                                 2 MiB (2
instances)\nL3 cache:                                  64 MiB (2 instances)\nNUMA
node(s):                          1\nNUMA node0 CPU(s):
0,1\nVulnerability Gather data sampling:      Not affected\nVulnerability Indirect
target selection: Not affected\nVulnerability Itlb multihit:             Not
affected\nVulnerability L1tf:                       Not affected\nVulnerability
Mds:                      Not affected\nVulnerability Meltdown:
Not affected\nVulnerability Mmio stale data:           Not affected\nVulnerability
Reg file data sampling:    Not affected\nVulnerability Retbleed:
Not affected\nVulnerability Spec rstack overflow:      Not affected\nVulnerability
Spec store bypass:         Not affected\nVulnerability Spectre v1:
Mitigation; usercopy/swapgs barriers and __user pointer
sanitization\nVulnerability Spectre v2:                 Mitigation; Retpolines;
IBPB conditional; STIBP disabled; RSB filling; PBRSB-eIBRS Not affected; BHI Not
affected\nVulnerability Srbds:                      Not affected\nVulnerability
Tsa:                      Not affected\nVulnerability Tsx async abort:
Not affected\nVulnerability Vmscape:                    Not affected\n",
  "memory": "               total        used        free      shared  buff/cache
available\nMem:           3.8Gi       1.2Gi       1.3Gi       140Mi       1.3Gi
2.2Gi\nSwap:             0B          0B          0B\n",
  "master": "local[*]",
  "parallelism": 2,
  "shuffle_partitions": "8",
  "aqe": "false",
  "ui": "http://10.0.2.15:4040",
```

```
  "driver_memory": "1536m",
  "autobroadcast": "-1"
}
```

1 Подготовка Parquet

Staging и перевод в Parquet исключены из замеров. Дополнительно выводятся реальные входные партиции.

In [2]:

```
PARQUET = ROOT/"data"/"spark_input_parquet"
names =
["geolocation","order_items","products","sellers","product_category_name_translati
on"]
for name in names:
    read_csv(name,infer=True).write.mode("overwrite").parquet(str(PARQUET/name))
geo, items, products, sellers, translations =
[spark.read.parquet(str(PARQUET/name)) for name in names]
result["inputs"] = {name:
{"rows":df.count(),"partitions":df.rdd.getNumPartitions()} for name,df in
zip(names,[geo,items,products,sellers,translations])}
print(result["inputs"])
_, result["metrics"]["baseline"] = measured(spark,"P4_BASELINE",items.count)

assert [result["inputs"][name]["rows"] for name in names] ==
[1000163,112650,32951,3095,71]
assert products.select("product_id").distinct().count() == products.count()
assert sellers.select("seller_id").distinct().count() == sellers.count()
assert items.join(products,"product_id","left_anti").count() == 0
assert items.join(sellers,"seller_id","left_anti").count() == 0
```

```
[Stage 1:>                                                          (0 + 2) / 2]
```

```
[Stage 2:>                                                          (0 + 2) / 2]
```

```
{'geolocation': {'rows': 1000163, 'partitions': 2}, 'order_items': {'rows':
112650, 'partitions': 2}, 'products': {'rows': 32951, 'partitions': 1}, 'sellers':
{'rows': 3095, 'partitions': 1}, 'product_category_name_translation': {'rows': 71,
'partitions': 1}}

P4_BASELINE {"seconds": 0.042502808006247506, "jobs": [25], "stages": [30, 31],
"tasks": 3, "shuffle_read": 118, "shuffle_write": 118, "stage_metrics":
[{"stageId": 31, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 118, "shuffleWriteBytes": 0}, {"stageId":
30, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 21482, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 118}]}
```

2 Lazy evaluation

Фиксируются Job ID перед созданием цепочки, после transformations и после count. Построение плана не должно запускать вычислительную Job.

In [3]:

```
sc.setJobGroup("P4_LAZY","Lazy evaluation")
before = sorted(sc.statusTracker().getJobIdsForGroup("P4_LAZY"))
prepared =
(geo.select("geolocation_zip_code_prefix","geolocation_state","geolocation_lat","g
eolocation_lng")

.filter(F.col("geolocation_state").isNotNull()).withColumn("abs_lat",F.abs("geoloc
ation_lat")))
after_transform = sorted(sc.statusTracker().getJobIdsForGroup("P4_LAZY"))
n, metric = measured(spark,"P4_LAZY",prepared.count)
result["lazy"] =
{"before":before,"after_transform":after_transform,"after_action":metric["jobs"],"
rows":n,"seconds":metric["seconds"]}
assert before==after_transform
print(result["lazy"])
```

```
P4_LAZY {"seconds": 0.0682418699725531, "jobs": [32], "stages": [50, 51], "tasks":
3, "shuffle_read": 118, "shuffle_write": 118, "stage_metrics": [{"stageId": 51,
"numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0, "outputBytes": 0,
"shuffleReadBytes": 118, "shuffleWriteBytes": 0}, {"stageId": 50, "numTasks": 2,
"numCompleteTasks": 2, "inputBytes": 45110, "outputBytes": 0, "shuffleReadBytes":
0, "shuffleWriteBytes": 118}]}
{'before': [], 'after_transform': [], 'after_action': [32], 'rows': 1000163,
'seconds': 0.0682418699725531}
```

3 Партиции и перекос

Диагностическая группировка сама является отдельным вычислением. Размеры партиций сравниваются на одинаковом наборе.

In [4]:

```
def profile(df):
    p =
df.select(F.spark_partition_id().alias("pid")).groupBy("pid").count().orderBy("pid
")
    values=records(p)
    npart=df.rdd.getNumPartitions()
    observed={row["pid"]:row["count"] for row in values}
    values=[{"pid":pid,"count":observed.get(pid,0)} for pid in range(npart)]
    sizes=[r["count"] for r in values]
    return
{"partitions":npart,"sizes":values,"min":min(sizes),"max":max(sizes),"mean":sum(si
zes)/len(sizes)}
state_counts=geo.groupBy("geolocation_state").count().orderBy(F.desc("count"))
state_counts.show(5,False)
result["state_top"]=records(state_counts.limit(5))
result["input_profile"]=profile(geo)
skewed16=geo.repartition(16,"geolocation_state")
repart4=skewed16.repartition(4)
coal4=skewed16.coalesce(4)
plan(repart4,"repartition4")
plan(coal4,"coalesce4")
result["profiles"]=
{"skewed16":profile(skewed16),"repartition4":profile(repart4),"coalesce4":profile(
coal4)}
_,result["metrics"]["repartition4"]=measured(spark,"P4_REPARTITION",repart4.count)
```

```
_,result["metrics"]["coalesce4"]=measured(spark,"P4_COALESCE",coal4.count)
print(json.dumps(result["profiles"],indent=2))

+-----------------+------+
|geolocation_state|count |
+-----------------+------+
|SP               |404268|
|MG               |126336|
|RJ               |121169|
|RS               |61851 |
|PR               |57859 |
+-----------------+------+
only showing top 5 rows


== Physical Plan ==
Exchange (3)
+- * ColumnarToRow (2)
   +- Scan parquet  (1)


(1) Scan parquet
Output [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/geolocation]
ReadSchema:
struct<geolocation_zip_code_prefix:int,geolocation_lat:double,geolocation_lng:doub
le,geolocation_city:string,geolocation_state:string>

(2) ColumnarToRow [codegen id : 1]
Input [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]

(3) Exchange
Input [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]
Arguments: RoundRobinPartitioning(4), REPARTITION_BY_NUM, [plan_id=950]


== Physical Plan ==
Coalesce (4)
+- Exchange (3)
   +- * ColumnarToRow (2)
      +- Scan parquet  (1)


(1) Scan parquet
Output [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/geolocation]
ReadSchema:
struct<geolocation_zip_code_prefix:int,geolocation_lat:double,geolocation_lng:doub
le,geolocation_city:string,geolocation_state:string>

(2) ColumnarToRow [codegen id : 1]
Input [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]
```

```
(3) Exchange
Input [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]
Arguments: hashpartitioning(geolocation_state#197, 16), REPARTITION_BY_NUM,
[plan_id=973]

(4) Coalesce
Input [5]: [geolocation_zip_code_prefix#193, geolocation_lat#194,
geolocation_lng#195, geolocation_city#196, geolocation_state#197]
Arguments: 4
```


```
P4_REPARTITION {"seconds": 0.3393313780252356, "jobs": [43], "stages": [84, 85,
86, 87], "tasks": 23, "shuffle_read": 176717, "shuffle_write": 176717,
"stage_metrics": [{"stageId": 87, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 236, "shuffleWriteBytes":
0}, {"stageId": 86, "numTasks": 4, "numCompleteTasks": 4, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 21232, "shuffleWriteBytes": 236},
{"stageId": 85, "numTasks": 16, "numCompleteTasks": 16, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 155249, "shuffleWriteBytes": 21232},
{"stageId": 84, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 34342,
"outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 155249}]}

P4_COALESCE {"seconds": 0.19001309599843808, "jobs": [44], "stages": [88, 89, 90],
"tasks": 7, "shuffle_read": 155485, "shuffle_write": 155485, "stage_metrics":
[{"stageId": 90, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 236, "shuffleWriteBytes": 0}, {"stageId":
89, "numTasks": 4, "numCompleteTasks": 4, "inputBytes": 0, "outputBytes": 0,
"shuffleReadBytes": 155249, "shuffleWriteBytes": 236}, {"stageId": 88, "numTasks":
2, "numCompleteTasks": 2, "inputBytes": 34342, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 155249}]}
{
  "skewed16": {
    "partitions": 16,
    "sizes": [
      {
        "pid": 0,
        "count": 40594
      },
      {
        "pid": 1,
        "count": 0
      },
      {
        "pid": 2,
        "count": 0
      },
      {
        "pid": 3,
        "count": 154470
      },
      {
        "pid": 4,
        "count": 466119
      },
      {
        "pid": 5,
        "count": 121815
      },
```

```
    {
      "pid": 6,
      "count": 12863
    },
    {
      "pid": 7,
      "count": 25688
    },
    {
      "pid": 8,
      "count": 1301
    },
    {
      "pid": 9,
      "count": 0
    },
    {
      "pid": 10,
      "count": 57859
    },
    {
      "pid": 11,
      "count": 30992
    },
    {
      "pid": 12,
      "count": 38328
    },
    {
      "pid": 13,
      "count": 0
    },
    {
      "pid": 14,
      "count": 22286
    },
    {
      "pid": 15,
      "count": 27848
    }
  ],
  "min": 0,
  "max": 466119,
  "mean": 62510.1875
},
"repartition4": {
  "partitions": 4,
  "sizes": [
    {
      "pid": 0,
      "count": 250040
    },
    {
      "pid": 1,
      "count": 250041
    },
    {
      "pid": 2,
      "count": 250041
    },
    {
```

```
        "pid": 3,
        "count": 250041
      }
    ],
    "min": 250040,
    "max": 250041,
    "mean": 250040.75
  },
  "coalesce4": {
    "partitions": 4,
    "sizes": [
      {
        "pid": 0,
        "count": 195064
      },
      {
        "pid": 1,
        "count": 626485
      },
      {
        "pid": 2,
        "count": 90152
      },
      {
        "pid": 3,
        "count": 88462
      }
    ],
    "min": 88462,
    "max": 626485,
    "mean": 250040.75
  }
}
```


4 GroupBy и physical plan

Частичная HashAggregate сокращает данные перед Exchange; финальная агрегация получает восемь shuffle partitions.

In [5]:

```
seller_stats=items.groupBy("seller_id").agg(F.count("*").alias("items"),F.round(F.
sum("price"),2).alias("revenue"),F.round(F.avg("price"),2).alias("avg_price")).ord
erBy(F.desc("revenue"))
top, result["metrics"]["groupby"] =
measured(spark,"P4_GROUPBY",lambda:records(seller_stats.limit(5)))
result["seller_top"]=top
seller_stats.show(5,False)
plan(seller_stats,"groupby")
demo=
(items.filter("price>=100").select("seller_id","price","freight_value").withColumn
("gross",F.col("price")+F.col("freight_value"))
    .groupBy("seller_id").agg(F.round(F.sum("gross"),2).alias("gross_sum")))
plan(demo,"operator_demo")
```


```
P4_GROUPBY {"seconds": 0.1828344019886572, "jobs": [45], "stages": [91, 92],
"tasks": 10, "shuffle_read": 269271, "shuffle_write": 269271, "stage_metrics":
[{"stageId": 92, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 269271, "shuffleWriteBytes": 0}, {"stageId":
```

```
91, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 819626, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 269271}]}
+--------------------------------+-----+---------+---------+
|seller_id                       |items|revenue  |avg_price|
+--------------------------------+-----+---------+---------+
|4869f7a5dfa277a7dca6462dcf3b52b2|1156 |229472.63|198.51   |
|53243585a1d6dc2643021fd1853d8905|410  |222776.05|543.36   |
|4a3ca9315b744ce9f8e9374361493884|1987 |200472.92|100.89   |
|fa1c13f2614d7b5c4749cbc52fecda94|586  |194042.03|331.13   |
|7c67e1448b00f6e969d365cea6b010ab|1364 |187923.89|137.77   |
+--------------------------------+-----+---------+---------+
only showing top 5 rows

== Physical Plan ==
* Sort (7)
+- Exchange (6)
   +- * HashAggregate (5)
      +- Exchange (4)
         +- * HashAggregate (3)
            +- * ColumnarToRow (2)
               +- Scan parquet  (1)


(1) Scan parquet
Output [2]: [seller_id#206, price#208]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
ReadSchema: struct<seller_id:string,price:double>

(2) ColumnarToRow [codegen id : 1]
Input [2]: [seller_id#206, price#208]

(3) HashAggregate [codegen id : 1]
Input [2]: [seller_id#206, price#208]
Keys [1]: [seller_id#206]
Functions [3]: [partial_count(1), partial_sum(price#208), partial_avg(price#208)]
Aggregate Attributes [4]: [count#503L, sum#504, sum#505, count#506L]
Results [5]: [seller_id#206, count#507L, sum#508, sum#509, count#510L]

(4) Exchange
Input [5]: [seller_id#206, count#507L, sum#508, sum#509, count#510L]
Arguments: hashpartitioning(seller_id#206, 8), ENSURE_REQUIREMENTS, [plan_id=1476]

(5) HashAggregate [codegen id : 2]
Input [5]: [seller_id#206, count#507L, sum#508, sum#509, count#510L]
Keys [1]: [seller_id#206]
Functions [3]: [count(1), sum(price#208), avg(price#208)]
Aggregate Attributes [3]: [count(1)#493L, sum(price#208)#495, avg(price#208)#497]
Results [4]: [seller_id#206, count(1)#493L AS items#494L,
round(sum(price#208)#495, 2) AS revenue#496, round(avg(price#208)#497, 2) AS
avg_price#498]

(6) Exchange
Input [4]: [seller_id#206, items#494L, revenue#496, avg_price#498]
Arguments: rangepartitioning(revenue#496 DESC NULLS LAST, 8), ENSURE_REQUIREMENTS,
[plan_id=1480]

(7) Sort [codegen id : 3]
Input [4]: [seller_id#206, items#494L, revenue#496, avg_price#498]
Arguments: [revenue#496 DESC NULLS LAST], true, 0
```

```
== Physical Plan ==
* HashAggregate (7)
+- Exchange (6)
   +- * HashAggregate (5)
      +- * Project (4)
         +- * Filter (3)
            +- * ColumnarToRow (2)
               +- Scan parquet  (1)


(1) Scan parquet
Output [3]: [seller_id#206, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThanOrEqual(price,100.0)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 1]
Input [3]: [seller_id#206, price#208, freight_value#209]

(3) Filter [codegen id : 1]
Input [3]: [seller_id#206, price#208, freight_value#209]
Condition : (isnotnull(price#208) AND (price#208 >= 100.0))

(4) Project [codegen id : 1]
Output [2]: [seller_id#206, (price#208 + freight_value#209) AS gross#546]
Input [3]: [seller_id#206, price#208, freight_value#209]

(5) HashAggregate [codegen id : 1]
Input [2]: [seller_id#206, gross#546]
Keys [1]: [seller_id#206]
Functions [1]: [partial_sum(gross#546)]
Aggregate Attributes [1]: [sum#559]
Results [2]: [seller_id#206, sum#560]

(6) Exchange
Input [2]: [seller_id#206, sum#560]
Arguments: hashpartitioning(seller_id#206, 8), ENSURE_REQUIREMENTS, [plan_id=1526]

(7) HashAggregate [codegen id : 2]
Input [2]: [seller_id#206, sum#560]
Keys [1]: [seller_id#206]
Functions [1]: [sum(gross#546)]
Aggregate Attributes [1]: [sum(gross#546)#555]
Results [2]: [seller_id#206, round(sum(gross#546)#555, 2) AS gross_sum#556]



'== Physical Plan ==\n* HashAggregate (7)\n+- Exchange (6)\n   +- * HashAggregate
(5)\n      +- * Project (4)\n         +- * Filter (3)\n            +- *
ColumnarToRow (2)\n               +- Scan parquet  (1)\n\n\n(1) Scan parquet
\nOutput [3]: [seller_id#206, price#208, freight_value#209]\nBatched:
true\nLocation: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]\nPushedFilters:
[IsNotNull(price), GreaterThanOrEqual(price,100.0)]\nReadSchema:
struct<seller_id:string,price:double,freight_value:double>\n\n(2) ColumnarToRow
[codegen id : 1]\nInput [3]: [seller_id#206, price#208, freight_value#209]\n\n(3)
Filter [codegen id : 1]\nInput [3]: [seller_id#206, price#208,
```

```
freight_value#209]\nCondition : (isnotnull(price#208) AND (price#208 >=
100.0))\n\n(4) Project [codegen id : 1]\nOutput [2]: [seller_id#206, (price#208 +
freight_value#209) AS gross#546]\nInput [3]: [seller_id#206, price#208,
freight_value#209]\n\n(5) HashAggregate [codegen id : 1]\nInput [2]:
[seller_id#206, gross#546]\nKeys [1]: [seller_id#206]\nFunctions [1]:
[partial_sum(gross#546)]\nAggregate Attributes [1]: [sum#559]\nResults [2]:
[seller_id#206, sum#560]\n\n(6) Exchange\nInput [2]: [seller_id#206,
sum#560]\nArguments: hashpartitioning(seller_id#206, 8), ENSURE_REQUIREMENTS,
[plan_id=1526]\n\n(7) HashAggregate [codegen id : 2]\nInput [2]: [seller_id#206,
sum#560]\nKeys [1]: [seller_id#206]\nFunctions [1]: [sum(gross#546)]\nAggregate
Attributes [1]: [sum(gross#546)#555]\nResults [2]: [seller_id#206,
round(sum(gross#546)#555, 2) AS gross_sum#556]\n\n'
```

5 Sort Merge Join и Broadcast Hash Join

JOIN по product_id одинаков по смыслу. Сравниваются число строк, планы и фактические счётчики Spark UI. Дополнительно проводятся три прогретых повторения обеих стратегий.

In [6]:

```
smj=items.hint("merge").join(products.hint("merge"),"product_id","inner")
bhj=items.join(F.broadcast(products),"product_id","inner")
plan(smj,"smj")
plan(bhj,"bhj")
smj_n,result["metrics"]["smj"]=measured(spark,"P4_SMJ",smj.count)
bhj_n,result["metrics"]["bhj"]=measured(spark,"P4_BHJ",bhj.count)
assert smj_n==bhj_n==items.count()
join_runs={"smj":[],"bhj":[]}
for _ in range(3):
    for name,df in [("smj",smj),("bhj",bhj)]:
        t=perf_counter(); df.count(); join_runs[name].append(perf_counter()-t)
result["joins"]={"rows":smj_n,"runs":join_runs,"medians":{k:median(v) for k,v in
join_runs.items()}}
print(result["joins"])
checkpoint("pr4_joins",spark)
```

```
== Physical Plan ==
* Project (12)
+- * SortMergeJoin Inner (11)
   :- * Sort (5)
   :  +- Exchange (4)
   :     +- * Filter (3)
   :        +- * ColumnarToRow (2)
   :           +- Scan parquet  (1)
   +- * Sort (10)
      +- Exchange (9)
         +- * Filter (8)
            +- * ColumnarToRow (7)
               +- Scan parquet  (6)


(1) Scan parquet
Output [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(product_id)]
```

```
ReadSchema:
struct<order_id:string,order_item_id:int,product_id:string,seller_id:string,shippi
ng_limit_date:timestamp,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 1]
Input [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]

(3) Filter [codegen id : 1]
Input [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]
Condition : isnotnull(product_id#205)

(4) Exchange
Input [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]
Arguments: hashpartitioning(product_id#205, 8), ENSURE_REQUIREMENTS,
[plan_id=1588]

(5) Sort [codegen id : 2]
Input [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]
Arguments: [product_id#205 ASC NULLS FIRST], false, 0

(6) Scan parquet
Output [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/products]
PushedFilters: [IsNotNull(product_id)]
ReadSchema:
struct<product_id:string,product_category_name:string,product_name_lenght:int,prod
uct_description_lenght:int,product_photos_qty:int,product_weight_g:int,product_len
gth_cm:int,product_height_cm:int,product_width_cm:int>

(7) ColumnarToRow [codegen id : 3]
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]

(8) Filter [codegen id : 3]
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Condition : isnotnull(product_id#217)

(9) Exchange
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Arguments: hashpartitioning(product_id#217, 8), ENSURE_REQUIREMENTS,
[plan_id=1597]

(10) Sort [codegen id : 4]
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Arguments: [product_id#217 ASC NULLS FIRST], false, 0
```

```
(11) SortMergeJoin [codegen id : 5]
Left keys [1]: [product_id#205]
Right keys [1]: [product_id#217]
Join type: Inner
Join condition: None

(12) Project [codegen id : 5]
Output [15]: [product_id#205, order_id#203, order_item_id#204, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209, product_category_name#218,
product_name_lenght#219, product_description_lenght#220, product_photos_qty#221,
product_weight_g#222, product_length_cm#223, product_height_cm#224,
product_width_cm#225]
Input [16]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209, product_id#217,
product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]



== Physical Plan ==
* Project (9)
+- * BroadcastHashJoin Inner BuildRight (8)
   :- * Filter (3)
   :  +- * ColumnarToRow (2)
   :     +- Scan parquet  (1)
   +- BroadcastExchange (7)
      +- * Filter (6)
         +- * ColumnarToRow (5)
            +- Scan parquet  (4)


(1) Scan parquet
Output [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(product_id)]
ReadSchema:
struct<order_id:string,order_item_id:int,product_id:string,seller_id:string,shippi
ng_limit_date:timestamp,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 2]
Input [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]

(3) Filter [codegen id : 2]
Input [7]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209]
Condition : isnotnull(product_id#205)

(4) Scan parquet
Output [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/products]
PushedFilters: [IsNotNull(product_id)]
ReadSchema:
```

```
struct<product_id:string,product_category_name:string,product_name_lenght:int,prod
uct_description_lenght:int,product_photos_qty:int,product_weight_g:int,product_len
gth_cm:int,product_height_cm:int,product_width_cm:int>

(5) ColumnarToRow [codegen id : 1]
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]

(6) Filter [codegen id : 1]
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Condition : isnotnull(product_id#217)

(7) BroadcastExchange
Input [9]: [product_id#217, product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]
Arguments: HashedRelationBroadcastMode(List(input[0, string, false]),false),
[plan_id=1663]

(8) BroadcastHashJoin [codegen id : 2]
Left keys [1]: [product_id#205]
Right keys [1]: [product_id#217]
Join type: Inner
Join condition: None

(9) Project [codegen id : 2]
Output [15]: [product_id#205, order_id#203, order_item_id#204, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209, product_category_name#218,
product_name_lenght#219, product_description_lenght#220, product_photos_qty#221,
product_weight_g#222, product_length_cm#223, product_height_cm#224,
product_width_cm#225]
Input [16]: [order_id#203, order_item_id#204, product_id#205, seller_id#206,
shipping_limit_date#207, price#208, freight_value#209, product_id#217,
product_category_name#218, product_name_lenght#219,
product_description_lenght#220, product_photos_qty#221, product_weight_g#222,
product_length_cm#223, product_height_cm#224, product_width_cm#225]



P4_SMJ {"seconds": 0.2429042930016294, "jobs": [47], "stages": [95, 96, 97, 98],
"tasks": 12, "shuffle_read": 3917308, "shuffle_write": 3917308, "stage_metrics":
[{"stageId": 98, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 472, "shuffleWriteBytes": 0}, {"stageId":
97, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0, "outputBytes": 0,
"shuffleReadBytes": 3916836, "shuffleWriteBytes": 472}, {"stageId": 96,
"numTasks": 1, "numCompleteTasks": 1, "inputBytes": 1141848, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 1114450}, {"stageId": 95, "numTasks":
2, "numCompleteTasks": 2, "inputBytes": 1687040, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 2802386}]}

P4_BHJ {"seconds": 0.22593405499355868, "jobs": [48, 49], "stages": [99, 100,
101], "tasks": 4, "shuffle_read": 118, "shuffle_write": 118, "stage_metrics":
[{"stageId": 101, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 118, "shuffleWriteBytes": 0}, {"stageId":
100, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 1687040, "outputBytes":
0, "shuffleReadBytes": 0, "shuffleWriteBytes": 118}, {"stageId": 99, "numTasks":
```

```
1, "numCompleteTasks": 1, "inputBytes": 1141848, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}

{'rows': 112650, 'runs': {'smj': [0.1776128570199944, 0.1531599269947037,
0.14379980898229405], 'bhj': [0.13414506299886853, 0.09413208201294765,
0.08230454399017617]}, 'medians': {'smj': 0.1531599269947037, 'bhj':
0.09413208201294765}}

AUTOMATIC UI CAPTURE pr4_joins 4
```

6 Persist и повторные действия

Сравнение включает цену материализации. Действия вычисляют контрольные суммы, чтобы оптимизатор не выбросил вычисляемое поле.

In [7]:

```
spark.catalog.clearCache()
joined=
(items.join(F.broadcast(products.select("product_id","product_category_name")),"pr
oduct_id","left")

.join(F.broadcast(sellers.select("seller_id","seller_state")),"seller_id","left")
    .withColumn("gross_value",F.col("price")+F.col("freight_value")))
def checksum(df,key):
    return
df.groupBy(key).agg(F.sum("gross_value").alias("gross")).agg(F.sum("gross").alias(
"checksum")).first()[0]
a,m1=measured(spark,"P4_NOCACHE",lambda:checksum(joined,"product_category_name"))
b,m2=measured(spark,"P4_NOCACHE",lambda:checksum(joined,"seller_state"))
cached=joined.persist(StorageLevel.MEMORY_AND_DISK)
cached_n,mat=measured(spark,"P4_PERSIST",cached.count)
c,m3=measured(spark,"P4_PERSIST",lambda:checksum(cached,"product_category_name"))
d,m4=measured(spark,"P4_PERSIST",lambda:checksum(cached,"seller_state"))
assert abs(a-c)<0.01 and abs(b-d)<0.01
result["cache"]={"rows":cached_n,"no_cache":
[m1,m2],"materialization":mat,"cached":[m3,m4],"checksums":
[a,b,c,d],"storage":ui_data(spark,"/storage/rdd")}
print(json.dumps(result["cache"],indent=2))
result["metrics"]["persist_materialization"]=mat
result["metrics"]["persist_repeat"]=m3
checkpoint("pr4_storage",spark,["/storage/","/SQL/","/stages/"])
cached.unpersist(blocking=True)
result["cache"]["after_unpersist"]=
{"is_cached":cached.is_cached,"storage":ui_data(spark,"/storage/rdd")}
assert not cached.is_cached and not result["cache"]["after_unpersist"]["storage"]
print("Cached after unpersist:",cached.is_cached)

P4_NOCACHE {"seconds": 0.3134857699915301, "jobs": [60, 61, 62], "stages": [125,
126, 127, 128, 129], "tasks": 13, "shuffle_read": 6445, "shuffle_write": 6445,
"stage_metrics": [{"stageId": 129, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 472, "shuffleWriteBytes":
0}, {"stageId": 128, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 5973, "shuffleWriteBytes": 472}, {"stageId":
127, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 2461575, "outputBytes":
0, "shuffleReadBytes": 0, "shuffleWriteBytes": 5973}, {"stageId": 126, "numTasks":
1, "numCompleteTasks": 1, "inputBytes": 131816, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}, {"stageId": 125, "numTasks": 1,
```

"numCompleteTasks": 1, "inputBytes": 1141848, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}

P4_NOCACHE {"seconds": 0.25310376900597475, "jobs": [63, 64, 65], "stages": [130,
131, 132, 133, 134], "tasks": 13, "shuffle_read": 2088, "shuffle_write": 2088,
"stage_metrics": [{"stageId": 134, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 470, "shuffleWriteBytes":
0}, {"stageId": 133, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 1618, "shuffleWriteBytes": 470}, {"stageId":
132, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 2461575, "outputBytes":
0, "shuffleReadBytes": 0, "shuffleWriteBytes": 1618}, {"stageId": 131, "numTasks":
1, "numCompleteTasks": 1, "inputBytes": 131816, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}, {"stageId": 130, "numTasks": 1,
"numCompleteTasks": 1, "inputBytes": 1141848, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}

P4_PERSIST {"seconds": 0.5269359890080523, "jobs": [66, 67, 68], "stages": [135,
136, 137, 138], "tasks": 5, "shuffle_read": 118, "shuffle_write": 118,
"stage_metrics": [{"stageId": 138, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 118, "shuffleWriteBytes":
0}, {"stageId": 137, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 6677415,
"outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 118}, {"stageId":
136, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 131816, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}, {"stageId": 135, "numTasks": 1,
"numCompleteTasks": 1, "inputBytes": 1141848, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}
P4_PERSIST {"seconds": 0.14940963900880888, "jobs": [69], "stages": [139, 140,
141], "tasks": 11, "shuffle_read": 6445, "shuffle_write": 6445, "stage_metrics":
[{"stageId": 141, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 472, "shuffleWriteBytes": 0}, {"stageId":
140, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0, "outputBytes": 0,
"shuffleReadBytes": 5973, "shuffleWriteBytes": 472}, {"stageId": 139, "numTasks":
2, "numCompleteTasks": 2, "inputBytes": 8727795, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 5973}]}

P4_PERSIST {"seconds": 0.0786903279949911, "jobs": [70], "stages": [142, 143,
144], "tasks": 11, "shuffle_read": 2088, "shuffle_write": 2088, "stage_metrics":
[{"stageId": 144, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 470, "shuffleWriteBytes": 0}, {"stageId":
143, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0, "outputBytes": 0,
"shuffleReadBytes": 1618, "shuffleWriteBytes": 470}, {"stageId": 142, "numTasks":
2, "numCompleteTasks": 2, "inputBytes": 8727795, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 1618}]}
{
  "rows": 112650,
  "no_cache": [
    {
      "seconds": 0.3134857699915301,
      "jobs": [
        60,
        61,
        62
      ],
      "stages": [
        125,
        126,
        127,
        128,
        129
      ],
      "tasks": 13,

```
  "shuffle_read": 6445,
  "shuffle_write": 6445,
  "stage_metrics": [
    {
      "stageId": 129,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 472,
      "shuffleWriteBytes": 0
    },
    {
      "stageId": 128,
      "numTasks": 8,
      "numCompleteTasks": 8,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 5973,
      "shuffleWriteBytes": 472
    },
    {
      "stageId": 127,
      "numTasks": 2,
      "numCompleteTasks": 2,
      "inputBytes": 2461575,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 5973
    },
    {
      "stageId": 126,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 131816,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 0
    },
    {
      "stageId": 125,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 1141848,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 0
    }
  ]
},
{
  "seconds": 0.25310376900597475,
  "jobs": [
    63,
    64,
    65
  ],
  "stages": [
    130,
    131,
    132,
```

```
      133,
      134
    ],
    "tasks": 13,
    "shuffle_read": 2088,
    "shuffle_write": 2088,
    "stage_metrics": [
      {
        "stageId": 134,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 0,
        "outputBytes": 0,
        "shuffleReadBytes": 470,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 133,
        "numTasks": 8,
        "numCompleteTasks": 8,
        "inputBytes": 0,
        "outputBytes": 0,
        "shuffleReadBytes": 1618,
        "shuffleWriteBytes": 470
      },
      {
        "stageId": 132,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 2461575,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 1618
      },
      {
        "stageId": 131,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 131816,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 130,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 1141848,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 0
      }
    ]
  }
],
"materialization": {
  "seconds": 0.5269359890080523,
  "jobs": [
    66,
    67,
    68
```

```
    ],
    "stages": [
      135,
      136,
      137,
      138
    ],
    "tasks": 5,
    "shuffle_read": 118,
    "shuffle_write": 118,
    "stage_metrics": [
      {
        "stageId": 138,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 0,
        "outputBytes": 0,
        "shuffleReadBytes": 118,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 137,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 6677415,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 118
      },
      {
        "stageId": 136,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 131816,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 135,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 1141848,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 0
      }
    ]
  },
  "cached": [
    {
      "seconds": 0.14940963900880888,
      "jobs": [
        69
      ],
      "stages": [
        139,
        140,
        141
      ],
      "tasks": 11,
```

```
  "shuffle_read": 6445,
  "shuffle_write": 6445,
  "stage_metrics": [
    {
      "stageId": 141,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 472,
      "shuffleWriteBytes": 0
    },
    {
      "stageId": 140,
      "numTasks": 8,
      "numCompleteTasks": 8,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 5973,
      "shuffleWriteBytes": 472
    },
    {
      "stageId": 139,
      "numTasks": 2,
      "numCompleteTasks": 2,
      "inputBytes": 8727795,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 5973
    }
  ]
},
{
  "seconds": 0.0786903279949911,
  "jobs": [
    70
  ],
  "stages": [
    142,
    143,
    144
  ],
  "tasks": 11,
  "shuffle_read": 2088,
  "shuffle_write": 2088,
  "stage_metrics": [
    {
      "stageId": 144,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 470,
      "shuffleWriteBytes": 0
    },
    {
      "stageId": 143,
      "numTasks": 8,
      "numCompleteTasks": 8,
      "inputBytes": 0,
      "outputBytes": 0,
```

```
          "shuffleReadBytes": 1618,
          "shuffleWriteBytes": 470
        },
        {
          "stageId": 142,
          "numTasks": 2,
          "numCompleteTasks": 2,
          "inputBytes": 8727795,
          "outputBytes": 0,
          "shuffleReadBytes": 0,
          "shuffleWriteBytes": 1618
        }
      ]
    }
  ],
  "checksums": [
    15843553.240000011,
    15843553.239999592,
    15843553.240000011,
    15843553.239999592
  ],
  "storage": [
    {
      "id": 546,
      "name": "*(3) Project [seller_id#206, product_id#205, order_id#203,
order_item_id#204, shipping_limit_date#207, price#208, freight_value#209,
product_category_name#218, seller_state#238, (price#208 + freight_value#209) AS
gross_value#785]\n+- *(3) BroadcastHashJoin [seller_id#206], [seller_id#235],
LeftOuter, BuildRight, false\n   :- *(3) Project [product_id#205, order_id#203,
order_item_id#204, seller_id#206, shipping_limit_date#207, price#208,
freight_value#209, product_category_name#218]\n   :  +- *(3) BroadcastHashJoin
[product_id#205], [product_id#217], LeftOuter, BuildRight, false\n   :     :- *(3)
ColumnarToRow\n   :     :  +- FileScan parquet
[order_id#203,order_item_id#204,product_id#205,seller_id#206,shipping_limit_date#2
07,price#208,freight_value#209] Batched: true, DataFilters: [], Format: Parquet,
Location: InMemoryFileIndex(1 paths)
[file:/home/albakhtin/tiabd/\u0410\u043b\u0431\u0430\u0445\u0442\u0438\u043d\u0418
\u0412/Spark/data/spark_input_parquet/o..., PartitionFilters: [], PushedFilters:
[], ReadSchema: struct<order_id:string,order_item_id:int,product...",
      "numPartitions": 2,
      "numCachedPartitions": 2,
      "storageLevel": "Disk Memory Serialized 1x Replicated",
      "memoryUsed": 8727795,
      "diskUsed": 0,
      "dataDistribution": [
        {
          "address": "10.0.2.15:41997",
          "memoryUsed": 8727795,
          "memoryRemaining": 742431265,
          "diskUsed": 0,
          "onHeapMemoryUsed": 8727795,
          "offHeapMemoryUsed": 0,
          "onHeapMemoryRemaining": 742431265,
          "offHeapMemoryRemaining": 0
        }
      ],
      "partitions": [
        {
          "blockName": "rdd_546_1",
          "storageLevel": "Memory Serialized 1x Replicated",
          "memoryUsed": 3151074,
```

```
          "diskUsed": 0,
          "executors": [
            "driver"
          ]
        },
        {
          "blockName": "rdd_546_0",
          "storageLevel": "Memory Serialized 1x Replicated",
          "memoryUsed": 5576721,
          "diskUsed": 0,
          "executors": [
            "driver"
          ]
        }
      ]
    }
  ]
}

AUTOMATIC UI CAPTURE pr4_storage 1

Cached after unpersist: False
```

7 Benchmark

N=2 000 000, key=id % 1000, value=sqrt(id+1). После одного прогрева для каждого p выполняются три запуска; число shuffle partitions неизменно и равно восьми.

In [8]:

```
N=2_000_000
def run_benchmark(p,repeats=3):
    times=[]; checks=[]
    for _ in range(repeats):
        df=spark.range(0,N,1,numPartitions=p).withColumn("key",
(F.col("id")%1000).cast("int")).withColumn("value",F.sqrt(F.col("id")+1.0))
        t=perf_counter()

check=df.groupBy("key").agg(F.sum("value").alias("s")).agg(F.sum("s")).first()[0]
        times.append(perf_counter()-t); checks.append(check)
    return {"p":p,"times":times,"median":median(times),"checksum":checks}
sc.setJobGroup("P4_BENCHMARK","Synthetic partition benchmark")
run_benchmark(4,1)
result["benchmark"]=[run_benchmark(p) for p in [1,2,4,8,16,32]]
for row in result["benchmark"]: row["speedup"]=result["benchmark"][0]
["median"]/row["median"]
print(json.dumps(result["benchmark"],indent=2))
geo8=geo.repartition(8)
q_results=[]
for q in [2,4,8,16,32]:
    spark.conf.set("spark.sql.shuffle.partitions",str(q))
    t=perf_counter(); rows=geo8.groupBy("geolocation_state").count().count();
seconds=perf_counter()-t
    q_results.append({"q":q,"groups":rows,"seconds":seconds})
spark.conf.set("spark.sql.shuffle.partitions","8")
result["shuffle_q"]=q_results
print(q_results)
```

```
[
  {
    "p": 1,
    "times": [
      0.09869962799712084,
      0.07022893402609043,
      0.050510408997070044
    ],
    "median": 0.07022893402609043,
    "checksum": [
      1885618790.063052,
      1885618790.063052,
      1885618790.063052
    ],
    "speedup": 1.0
  },
  {
    "p": 2,
    "times": [
      0.12054276000708342,
      0.08861396298743784,
      0.08302843300043605
    ],
    "median": 0.08861396298743784,
    "checksum": [
      1885618790.0630507,
      1885618790.0630507,
      1885618790.0630507
    ],
    "speedup": 0.7925267266970812
  },
  {
    "p": 4,
    "times": [
      0.09255835198564455,
      0.07823761197505519,
      0.08059796501765959
    ],
    "median": 0.08059796501765959,
    "checksum": [
      1885618790.0630512,
      1885618790.0630512,
      1885618790.0630512
    ],
    "speedup": 0.8713487246322257
  },
  {
    "p": 8,
    "times": [
      0.14336307501071133,
      0.13876656498177908,
      0.0784745079872664
    ],
    "median": 0.13876656498177908,
    "checksum": [
      1885618790.0630512,
      1885618790.0630512,
      1885618790.0630512
    ],
    "speedup": 0.5060940582863885
  },
```

```
  {
    "p": 16,
    "times": [
      0.14992408498073928,
      0.09387935898848809,
      0.09810651099542156
    ],
    "median": 0.09810651099542156,
    "checksum": [
      1885618790.0630512,
      1885618790.0630512,
      1885618790.0630512
    ],
    "speedup": 0.715843763207193
  },
  {
    "p": 32,
    "times": [
      0.16136933700181544,
      0.13553735698224045,
      0.13801017799414694
    ],
    "median": 0.13801017799414694,
    "checksum": [
      1885618790.0630512,
      1885618790.0630512,
      1885618790.0630512
    ],
    "speedup": 0.5088677882081194
  }
]
```

```
[{'q': 2, 'groups': 27, 'seconds': 0.29883613900165074}, {'q': 4, 'groups': 27,
'seconds': 0.2214604360051453}, {'q': 8, 'groups': 27, 'seconds':
0.24436699101352133}, {'q': 16, 'groups': 27, 'seconds': 0.27139667799929157},
{'q': 32, 'groups': 27, 'seconds': 0.2622285540273879}]
```


8 Маленькие файлы

Число файлов и их размер измеряются в файловой системе после фактической записи, а не предсказываются по числу партиций.

In [9]:

```
SMALL=ROOT/"small_files_experiment"
def write_measure(df,p,tag):
    path=SMALL/tag

_,metrics=measured(spark,"P4_FILES_"+tag,lambda:df.repartition(p).write.mode("over
write").parquet(str(path)))
    files=list(path.glob("part-*.parquet"))
    sizes=[f.stat().st_size for f in files]
    return
{"p":p,"files":len(files),"total_mib":sum(sizes)/1024**2,"mean_kib":sum(sizes)/len
(sizes)/1024,"metrics":metrics}
result["small_files"]=[write_measure(items,p,f"items_{p}") for p in [1,4,16,64]]
items.coalesce(1).write.mode("overwrite").parquet(str(SMALL/"coalesce1"))
print(json.dumps(result["small_files"],indent=2))
spark.conf.set("spark.sql.adaptive.enabled","true")
```

```
spark.conf.set("spark.sql.shuffle.partitions","32")
adaptive=geo.groupBy("geolocation_state").count()
adaptive.collect()
result["aqe"]=
{"groups":len(adaptive.collect()),"output_partitions":adaptive.rdd.getNumPartition
s()}
plan(adaptive,"aqe_final")
spark.conf.set("spark.sql.adaptive.enabled","false")
spark.conf.set("spark.sql.shuffle.partitions","8")
print(result["aqe"])
```

```
P4_FILES_items_1 {"seconds": 0.5224456729774829, "jobs": [95], "stages": [216,
217], "tasks": 3, "shuffle_read": 10751186, "shuffle_write": 10751186,
"stage_metrics": [{"stageId": 217, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 6770099, "shuffleReadBytes": 10751186,
"shuffleWriteBytes": 0}, {"stageId": 216, "numTasks": 2, "numCompleteTasks": 2,
"inputBytes": 6677415, "outputBytes": 0, "shuffleReadBytes": 0,
"shuffleWriteBytes": 10751186}]}

P4_FILES_items_4 {"seconds": 0.5558315539965406, "jobs": [96], "stages": [218,
219], "tasks": 6, "shuffle_read": 12012709, "shuffle_write": 12012709,
"stage_metrics": [{"stageId": 219, "numTasks": 4, "numCompleteTasks": 4,
"inputBytes": 0, "outputBytes": 7827036, "shuffleReadBytes": 12012709,
"shuffleWriteBytes": 0}, {"stageId": 218, "numTasks": 2, "numCompleteTasks": 2,
"inputBytes": 6677415, "outputBytes": 0, "shuffleReadBytes": 0,
"shuffleWriteBytes": 12012709}]}

P4_FILES_items_16 {"seconds": 0.7877706900180783, "jobs": [97], "stages": [220,
221], "tasks": 18, "shuffle_read": 12022984, "shuffle_write": 12022984,
"stage_metrics": [{"stageId": 221, "numTasks": 16, "numCompleteTasks": 16,
"inputBytes": 0, "outputBytes": 9181711, "shuffleReadBytes": 12022984,
"shuffleWriteBytes": 0}, {"stageId": 220, "numTasks": 2, "numCompleteTasks": 2,
"inputBytes": 6677415, "outputBytes": 0, "shuffleReadBytes": 0,
"shuffleWriteBytes": 12022984}]}

[Stage 223:=========================================>            (50 + 2) / 64]



P4_FILES_items_64 {"seconds": 0.9863997979846317, "jobs": [98], "stages": [222,
223], "tasks": 66, "shuffle_read": 12065531, "shuffle_write": 12065531,
"stage_metrics": [{"stageId": 223, "numTasks": 64, "numCompleteTasks": 64,
"inputBytes": 0, "outputBytes": 10801884, "shuffleReadBytes": 12065531,
"shuffleWriteBytes": 0}, {"stageId": 222, "numTasks": 2, "numCompleteTasks": 2,
"inputBytes": 6677415, "outputBytes": 0, "shuffleReadBytes": 0,
"shuffleWriteBytes": 12065531}]}
[
  {
    "p": 1,
    "files": 1,
    "total_mib": 6.456469535827637,
    "mean_kib": 6611.4248046875,
    "metrics": {
      "seconds": 0.5224456729774829,
      "jobs": [
        95
      ],
      "stages": [
        216,
        217
```

```
    ],
    "tasks": 3,
    "shuffle_read": 10751186,
    "shuffle_write": 10751186,
    "stage_metrics": [
      {
        "stageId": 217,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 0,
        "outputBytes": 6770099,
        "shuffleReadBytes": 10751186,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 216,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 6677415,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 10751186
      }
    ]
  }
},
{
  "p": 4,
  "files": 4,
  "total_mib": 7.464443206787109,
  "mean_kib": 1910.8974609375,
  "metrics": {
    "seconds": 0.5558315539965406,
    "jobs": [
      96
    ],
    "stages": [
      218,
      219
    ],
    "tasks": 6,
    "shuffle_read": 12012709,
    "shuffle_write": 12012709,
    "stage_metrics": [
      {
        "stageId": 219,
        "numTasks": 4,
        "numCompleteTasks": 4,
        "inputBytes": 0,
        "outputBytes": 7827036,
        "shuffleReadBytes": 12012709,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 218,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 6677415,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 12012709
```

```
      }
    ]
  }
},
{
  "p": 16,
  "files": 16,
  "total_mib": 8.756361961364746,
  "mean_kib": 560.4071655273438,
  "metrics": {
    "seconds": 0.7877706900180783,
    "jobs": [
      97
    ],
    "stages": [
      220,
      221
    ],
    "tasks": 18,
    "shuffle_read": 12022984,
    "shuffle_write": 12022984,
    "stage_metrics": [
      {
        "stageId": 221,
        "numTasks": 16,
        "numCompleteTasks": 16,
        "inputBytes": 0,
        "outputBytes": 9181711,
        "shuffleReadBytes": 12022984,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 220,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 6677415,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 12022984
      }
    ]
  }
},
{
  "p": 64,
  "files": 64,
  "total_mib": 10.30147933959961,
  "mean_kib": 164.82366943359375,
  "metrics": {
    "seconds": 0.9863997979846317,
    "jobs": [
      98
    ],
    "stages": [
      222,
      223
    ],
    "tasks": 66,
    "shuffle_read": 12065531,
    "shuffle_write": 12065531,
    "stage_metrics": [
```

```
        {
          "stageId": 223,
          "numTasks": 64,
          "numCompleteTasks": 64,
          "inputBytes": 0,
          "outputBytes": 10801884,
          "shuffleReadBytes": 12065531,
          "shuffleWriteBytes": 0
        },
        {
          "stageId": 222,
          "numTasks": 2,
          "numCompleteTasks": 2,
          "inputBytes": 6677415,
          "outputBytes": 0,
          "shuffleReadBytes": 0,
          "shuffleWriteBytes": 12065531
        }
      ]
    }
  }
]
```


```
== Physical Plan ==
AdaptiveSparkPlan (11)
+- == Final Plan ==
   * HashAggregate (7)
   +- AQEShuffleRead (6)
      +- ShuffleQueryStage (5), Statistics(sizeInBytes=960.0 B, rowCount=30)
         +- Exchange (4)
            +- * HashAggregate (3)
               +- * ColumnarToRow (2)
                  +- Scan parquet  (1)
+- == Initial Plan ==
   HashAggregate (10)
   +- Exchange (9)
      +- HashAggregate (8)
         +- Scan parquet  (1)


(1) Scan parquet
Output [1]: [geolocation_state#197]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/geolocation]
ReadSchema: struct<geolocation_state:string>

(2) ColumnarToRow [codegen id : 1]
Input [1]: [geolocation_state#197]

(3) HashAggregate [codegen id : 1]
Input [1]: [geolocation_state#197]
Keys [1]: [geolocation_state#197]
Functions [1]: [partial_count(1)]
Aggregate Attributes [1]: [count#2382L]
Results [2]: [geolocation_state#197, count#2383L]

(4) Exchange
Input [2]: [geolocation_state#197, count#2383L]
Arguments: hashpartitioning(geolocation_state#197, 32), ENSURE_REQUIREMENTS,
[plan_id=4470]
```

```
(5) ShuffleQueryStage
Output [2]: [geolocation_state#197, count#2383L]
Arguments: 0

(6) AQEShuffleRead
Input [2]: [geolocation_state#197, count#2383L]
Arguments: coalesced

(7) HashAggregate [codegen id : 2]
Input [2]: [geolocation_state#197, count#2383L]
Keys [1]: [geolocation_state#197]
Functions [1]: [count(1)]
Aggregate Attributes [1]: [count(1)#2378L]
Results [2]: [geolocation_state#197, count(1)#2378L AS count#2379L]

(8) HashAggregate
Input [1]: [geolocation_state#197]
Keys [1]: [geolocation_state#197]
Functions [1]: [partial_count(1)]
Aggregate Attributes [1]: [count#2382L]
Results [2]: [geolocation_state#197, count#2383L]

(9) Exchange
Input [2]: [geolocation_state#197, count#2383L]
Arguments: hashpartitioning(geolocation_state#197, 32), ENSURE_REQUIREMENTS,
[plan_id=4456]

(10) HashAggregate
Input [2]: [geolocation_state#197, count#2383L]
Keys [1]: [geolocation_state#197]
Functions [1]: [count(1)]
Aggregate Attributes [1]: [count(1)#2378L]
Results [2]: [geolocation_state#197, count(1)#2378L AS count#2379L]

(11) AdaptiveSparkPlan
Output [2]: [geolocation_state#197, count#2379L]
Arguments: isFinalPlan=true


{'groups': 27, 'output_partitions': 1}
```

9 Самостоятельный вариант B

order_items ↔ sellers, категориальный ключ seller_state. Повторяются lazy, repartition/coalesce, groupBy, две стратегии JOIN, persist, медианы и запись.

In [10]:

```
sc.setJobGroup("P4_B_LAZY","Independent B lazy")
before=sorted(sc.statusTracker().getJobIdsForGroup("P4_B_LAZY"))
b_prepared=items.select("seller_id","price","freight_value").filter("price>0").wit
hColumn("gross",F.col("price")+F.col("freight_value"))
after=sorted(sc.statusTracker().getJobIdsForGroup("P4_B_LAZY"))
b_rows,b_lazy=measured(spark,"P4_B_LAZY",b_prepared.count)
assert before==after
b_smj=b_prepared.hint("merge").join(sellers.hint("merge"),"seller_id","inner")
b_bhj=b_prepared.join(F.broadcast(sellers),"seller_id","inner")
plan(b_smj,"b_smj"); plan(b_bhj,"b_bhj")
b_sn,b_sm=measured(spark,"P4_B_SMJ",b_smj.count)
```

```
b_bn,b_bm=measured(spark,"P4_B_BHJ",b_bhj.count)
assert b_sn==b_bn==b_rows
b_group=b_bhj.groupBy("seller_state").agg(F.count("*").alias("lines"),F.round(F.su
m("gross"),2).alias("gross"))
b_values,b_gm=measured(spark,"P4_B_GROUPBY",lambda:records(b_group.orderBy(F.desc(
"gross"))))
plan(b_group,"b_groupby")
b_skew=b_bhj.repartition(16,"seller_state")
b_repart=b_skew.repartition(4); b_coal=b_skew.coalesce(4)
plan(b_repart,"b_repartition"); plan(b_coal,"b_coalesce")
b_profiles={"repartition":profile(b_repart),"coalesce":profile(b_coal)}
def b_action(df,key):
    return df.groupBy(key).agg(F.sum("gross").alias("s")).agg(F.sum("s")).first()
[0]
t=perf_counter(); ba=b_action(b_bhj,"seller_state"); t1=perf_counter();
bb=b_action(b_bhj,"seller_city"); t2=perf_counter()
b_cache=b_bhj.persist(StorageLevel.MEMORY_AND_DISK)
t3=perf_counter(); b_cache.count(); t4=perf_counter();
bc=b_action(b_cache,"seller_state"); t5=perf_counter();
bd=b_action(b_cache,"seller_city"); t6=perf_counter()
assert max(abs(ba-bc),abs(bb-bd))<0.01
b_cache_times={"without":[t1-t,t2-t1],"materialization":t4-t3,"cached":[t5-t4,t6-
t5]}
checkpoint("pr4_variant_storage",spark,["/storage/","/SQL/","/stages/"])
b_storage=ui_data(spark,"/storage/rdd")
b_cache.unpersist(blocking=True)
b_after_storage=ui_data(spark,"/storage/rdd")
assert not b_cache.is_cached and not b_after_storage
b_benchmark=[]
for p in [1,2,4,8]:
    times=[]
    for repeat in range(4):

variant=b_bhj.repartition(p).groupBy("seller_state").agg(F.sum("gross").alias("g")
)
        t=perf_counter(); values=variant.collect(); dt=perf_counter()-t
        if repeat: times.append(dt)
    b_benchmark.append({"p":p,"times":times,"median":median(times)})
b_files=[write_measure(b_group,p,f"variant_{p}") for p in [1,4]]
b_group.orderBy("seller_state").toPandas().to_csv(OUT/"pr4_variant.csv",index=Fals
e)
b_group.write.mode("overwrite").parquet(str(OUT/"pr4_variant_parquet"))
result["variant"]={"name":"B","inputs":{"items":result["inputs"]
["order_items"],"sellers":result["inputs"]["sellers"]},"lazy":
{"before":before,"after_transform":after,"after_action":b_lazy["jobs"],"rows":b_ro
ws},"join_rows":b_sn,"smj":b_sm,"bhj":b_bm,"groupby":b_gm,"groups":b_values,"profi
les":b_profiles,"cache":{**b_cache_times,"checksums":
[ba,bb,bc,bd],"storage":b_storage,"after_unpersist":b_after_storage},"benchmark":b
_benchmark,"files":b_files}
print(json.dumps(result["variant"],ensure_ascii=False,indent=2))

P4_B_LAZY {"seconds": 0.04312271700473502, "jobs": [103], "stages": [230, 231],
"tasks": 3, "shuffle_read": 118, "shuffle_write": 118, "stage_metrics":
[{"stageId": 231, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 118, "shuffleWriteBytes": 0}, {"stageId":
230, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 491124, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 118}]}
== Physical Plan ==
* Project (13)
+- * SortMergeJoin Inner (12)
   :- * Sort (6)
```

```
   :  +- Exchange (5)
   :     +- * Project (4)
   :        +- * Filter (3)
   :           +- * ColumnarToRow (2)
   :              +- Scan parquet  (1)
   +- * Sort (11)
      +- Exchange (10)
         +- * Filter (9)
            +- * ColumnarToRow (8)
               +- Scan parquet  (7)


(1) Scan parquet
Output [3]: [seller_id#206, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 1]
Input [3]: [seller_id#206, price#208, freight_value#209]

(3) Filter [codegen id : 1]
Input [3]: [seller_id#206, price#208, freight_value#209]
Condition : ((isnotnull(price#208) AND (price#208 > 0.0)) AND
isnotnull(seller_id#206))

(4) Project [codegen id : 1]
Output [4]: [seller_id#206, price#208, freight_value#209, (price#208 +
freight_value#209) AS gross#2387]
Input [3]: [seller_id#206, price#208, freight_value#209]

(5) Exchange
Input [4]: [seller_id#206, price#208, freight_value#209, gross#2387]
Arguments: hashpartitioning(seller_id#206, 8), ENSURE_REQUIREMENTS, [plan_id=4576]

(6) Sort [codegen id : 2]
Input [4]: [seller_id#206, price#208, freight_value#209, gross#2387]
Arguments: [seller_id#206 ASC NULLS FIRST], false, 0

(7) Scan parquet
Output [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/sellers]
PushedFilters: [IsNotNull(seller_id)]
ReadSchema:
struct<seller_id:string,seller_zip_code_prefix:int,seller_city:string,seller_state
:string>

(8) ColumnarToRow [codegen id : 3]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]

(9) Filter [codegen id : 3]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Condition : isnotnull(seller_id#235)
```

```
(10) Exchange
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Arguments: hashpartitioning(seller_id#235, 8), ENSURE_REQUIREMENTS, [plan_id=4585]

(11) Sort [codegen id : 4]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Arguments: [seller_id#235 ASC NULLS FIRST], false, 0

(12) SortMergeJoin [codegen id : 5]
Left keys [1]: [seller_id#206]
Right keys [1]: [seller_id#235]
Join type: Inner
Join condition: None

(13) Project [codegen id : 5]
Output [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Input [8]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_id#235, seller_zip_code_prefix#236, seller_city#237, seller_state#238]


== Physical Plan ==
* Project (10)
+- * BroadcastHashJoin Inner BuildRight (9)
   :- * Project (4)
   :  +- * Filter (3)
   :     +- * ColumnarToRow (2)
   :        +- Scan parquet  (1)
   +- BroadcastExchange (8)
      +- * Filter (7)
         +- * ColumnarToRow (6)
            +- Scan parquet  (5)


(1) Scan parquet
Output [3]: [seller_id#206, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]

(3) Filter [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]
Condition : ((isnotnull(price#208) AND (price#208 > 0.0)) AND
isnotnull(seller_id#206))

(4) Project [codegen id : 2]
Output [4]: [seller_id#206, price#208, freight_value#209, (price#208 +
freight_value#209) AS gross#2387]
Input [3]: [seller_id#206, price#208, freight_value#209]

(5) Scan parquet
Output [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Batched: true
```

```
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/sellers]
PushedFilters: [IsNotNull(seller_id)]
ReadSchema:
struct<seller_id:string,seller_zip_code_prefix:int,seller_city:string,seller_state
:string>

(6) ColumnarToRow [codegen id : 1]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]

(7) Filter [codegen id : 1]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Condition : isnotnull(seller_id#235)

(8) BroadcastExchange
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Arguments: HashedRelationBroadcastMode(List(input[0, string, false]),false),
[plan_id=4655]

(9) BroadcastHashJoin [codegen id : 2]
Left keys [1]: [seller_id#206]
Right keys [1]: [seller_id#235]
Join type: Inner
Join condition: None

(10) Project [codegen id : 2]
Output [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Input [8]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_id#235, seller_zip_code_prefix#236, seller_city#237, seller_state#238]



P4_B_SMJ {"seconds": 0.16632000298704952, "jobs": [104], "stages": [232, 233, 234,
235], "tasks": 12, "shuffle_read": 1369691, "shuffle_write": 1369691,
"stage_metrics": [{"stageId": 235, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 472, "shuffleWriteBytes":
0}, {"stageId": 234, "numTasks": 8, "numCompleteTasks": 8, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 1369219, "shuffleWriteBytes": 472},
{"stageId": 233, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 131816,
"outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 106379}, {"stageId":
232, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 964388, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 1262840}]}
P4_B_BHJ {"seconds": 0.09229239900014363, "jobs": [105, 106], "stages": [236, 237,
238], "tasks": 4, "shuffle_read": 118, "shuffle_write": 118, "stage_metrics":
[{"stageId": 238, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 0,
"outputBytes": 0, "shuffleReadBytes": 118, "shuffleWriteBytes": 0}, {"stageId":
237, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 964388, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 118}, {"stageId": 236, "numTasks": 1,
"numCompleteTasks": 1, "inputBytes": 131816, "outputBytes": 0, "shuffleReadBytes":
0, "shuffleWriteBytes": 0}]}

P4_B_GROUPBY {"seconds": 0.20214182301424444, "jobs": [107, 108, 109], "stages":
[239, 240, 241, 242, 243, 244], "tasks": 27, "shuffle_read": 5322,
"shuffle_write": 3472, "stage_metrics": [{"stageId": 244, "numTasks": 8,
"numCompleteTasks": 8, "inputBytes": 0, "outputBytes": 0, "shuffleReadBytes":
1622, "shuffleWriteBytes": 0}, {"stageId": 243, "numTasks": 8, "numCompleteTasks":
8, "inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 1850,
```

```
"shuffleWriteBytes": 1622}, {"stageId": 241, "numTasks": 8, "numCompleteTasks": 8,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 1850, "shuffleWriteBytes":
0}, {"stageId": 240, "numTasks": 2, "numCompleteTasks": 2, "inputBytes": 1086721,
"outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 1850}, {"stageId":
239, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 131816, "outputBytes": 0,
"shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}
== Physical Plan ==
* HashAggregate (13)
+- Exchange (12)
   +- * HashAggregate (11)
      +- * Project (10)
         +- * BroadcastHashJoin Inner BuildRight (9)
            :- * Project (4)
            :  +- * Filter (3)
            :     +- * ColumnarToRow (2)
            :        +- Scan parquet  (1)
            +- BroadcastExchange (8)
               +- * Filter (7)
                  +- * ColumnarToRow (6)
                     +- Scan parquet  (5)


(1) Scan parquet
Output [3]: [seller_id#206, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]

(3) Filter [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]
Condition : ((isnotnull(price#208) AND (price#208 > 0.0)) AND
isnotnull(seller_id#206))

(4) Project [codegen id : 2]
Output [2]: [seller_id#206, (price#208 + freight_value#209) AS gross#2387]
Input [3]: [seller_id#206, price#208, freight_value#209]

(5) Scan parquet
Output [2]: [seller_id#235, seller_state#238]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/sellers]
PushedFilters: [IsNotNull(seller_id)]
ReadSchema: struct<seller_id:string,seller_state:string>

(6) ColumnarToRow [codegen id : 1]
Input [2]: [seller_id#235, seller_state#238]

(7) Filter [codegen id : 1]
Input [2]: [seller_id#235, seller_state#238]
Condition : isnotnull(seller_id#235)

(8) BroadcastExchange
Input [2]: [seller_id#235, seller_state#238]
Arguments: HashedRelationBroadcastMode(List(input[0, string, false]),false),
[plan_id=5054]
```

```
(9) BroadcastHashJoin [codegen id : 2]
Left keys [1]: [seller_id#206]
Right keys [1]: [seller_id#235]
Join type: Inner
Join condition: None

(10) Project [codegen id : 2]
Output [2]: [gross#2387, seller_state#238]
Input [4]: [seller_id#206, gross#2387, seller_id#235, seller_state#238]

(11) HashAggregate [codegen id : 2]
Input [2]: [gross#2387, seller_state#238]
Keys [1]: [seller_state#238]
Functions [2]: [partial_count(1), partial_sum(gross#2387)]
Aggregate Attributes [2]: [count#2453L, sum#2454]
Results [3]: [seller_state#238, count#2455L, sum#2456]

(12) Exchange
Input [3]: [seller_state#238, count#2455L, sum#2456]
Arguments: hashpartitioning(seller_state#238, 8), ENSURE_REQUIREMENTS,
[plan_id=5060]

(13) HashAggregate [codegen id : 3]
Input [3]: [seller_state#238, count#2455L, sum#2456]
Keys [1]: [seller_state#238]
Functions [2]: [count(1), sum(gross#2387)]
Aggregate Attributes [2]: [count(1)#2446L, sum(gross#2387)#2448]
Results [3]: [seller_state#238, count(1)#2446L AS lines#2447L,
round(sum(gross#2387)#2448, 2) AS gross#2449]


== Physical Plan ==
Exchange (11)
+- * Project (10)
   +- * BroadcastHashJoin Inner BuildRight (9)
      :- * Project (4)
      :  +- * Filter (3)
      :     +- * ColumnarToRow (2)
      :        +- Scan parquet  (1)
      +- BroadcastExchange (8)
         +- * Filter (7)
            +- * ColumnarToRow (6)
               +- Scan parquet  (5)


(1) Scan parquet
Output [3]: [seller_id#206, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]

(3) Filter [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]
Condition : ((isnotnull(price#208) AND (price#208 > 0.0)) AND
isnotnull(seller_id#206))
```

```
(4) Project [codegen id : 2]
Output [4]: [seller_id#206, price#208, freight_value#209, (price#208 +
freight_value#209) AS gross#2387]
Input [3]: [seller_id#206, price#208, freight_value#209]

(5) Scan parquet
Output [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/sellers]
PushedFilters: [IsNotNull(seller_id)]
ReadSchema:
struct<seller_id:string,seller_zip_code_prefix:int,seller_city:string,seller_state
:string>

(6) ColumnarToRow [codegen id : 1]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]

(7) Filter [codegen id : 1]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Condition : isnotnull(seller_id#235)

(8) BroadcastExchange
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Arguments: HashedRelationBroadcastMode(List(input[0, string, false]),false),
[plan_id=5139]

(9) BroadcastHashJoin [codegen id : 2]
Left keys [1]: [seller_id#206]
Right keys [1]: [seller_id#235]
Join type: Inner
Join condition: None

(10) Project [codegen id : 2]
Output [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Input [8]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_id#235, seller_zip_code_prefix#236, seller_city#237, seller_state#238]

(11) Exchange
Input [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Arguments: RoundRobinPartitioning(4), REPARTITION_BY_NUM, [plan_id=5144]


== Physical Plan ==
Coalesce (12)
+- Exchange (11)
   +- * Project (10)
      +- * BroadcastHashJoin Inner BuildRight (9)
         :- * Project (4)
         :  +- * Filter (3)
         :     +- * ColumnarToRow (2)
         :        +- Scan parquet  (1)
         +- BroadcastExchange (8)
            +- * Filter (7)
```

```
               +- * ColumnarToRow (6)
                  +- Scan parquet  (5)


(1) Scan parquet
Output [3]: [seller_id#206, price#208, freight_value#209]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/order_items]
PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)]
ReadSchema: struct<seller_id:string,price:double,freight_value:double>

(2) ColumnarToRow [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]

(3) Filter [codegen id : 2]
Input [3]: [seller_id#206, price#208, freight_value#209]
Condition : ((isnotnull(price#208) AND (price#208 > 0.0)) AND
isnotnull(seller_id#206))

(4) Project [codegen id : 2]
Output [4]: [seller_id#206, price#208, freight_value#209, (price#208 +
freight_value#209) AS gross#2387]
Input [3]: [seller_id#206, price#208, freight_value#209]

(5) Scan parquet
Output [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Batched: true
Location: InMemoryFileIndex [file:/home/albakhtin/tiabd/
АлбахтинИВ/Spark/data/spark_input_parquet/sellers]
PushedFilters: [IsNotNull(seller_id)]
ReadSchema:
struct<seller_id:string,seller_zip_code_prefix:int,seller_city:string,seller_state
:string>

(6) ColumnarToRow [codegen id : 1]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]

(7) Filter [codegen id : 1]
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Condition : isnotnull(seller_id#235)

(8) BroadcastExchange
Input [4]: [seller_id#235, seller_zip_code_prefix#236, seller_city#237,
seller_state#238]
Arguments: HashedRelationBroadcastMode(List(input[0, string, false]),false),
[plan_id=5224]

(9) BroadcastHashJoin [codegen id : 2]
Left keys [1]: [seller_id#206]
Right keys [1]: [seller_id#235]
Join type: Inner
Join condition: None

(10) Project [codegen id : 2]
Output [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Input [8]: [seller_id#206, price#208, freight_value#209, gross#2387,
```

```
seller_id#235, seller_zip_code_prefix#236, seller_city#237, seller_state#238]

(11) Exchange
Input [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Arguments: hashpartitioning(seller_state#238, 16), REPARTITION_BY_NUM,
[plan_id=5229]

(12) Coalesce
Input [7]: [seller_id#206, price#208, freight_value#209, gross#2387,
seller_zip_code_prefix#236, seller_city#237, seller_state#238]
Arguments: 4



AUTOMATIC UI CAPTURE pr4_variant_storage 3

P4_FILES_variant_1 {"seconds": 0.15960440601338632, "jobs": [158, 159], "stages":
[342, 343, 344, 345], "tasks": 12, "shuffle_read": 2800, "shuffle_write": 2800,
"stage_metrics": [{"stageId": 345, "numTasks": 1, "numCompleteTasks": 1,
"inputBytes": 0, "outputBytes": 1367, "shuffleReadBytes": 950,
"shuffleWriteBytes": 0}, {"stageId": 344, "numTasks": 8, "numCompleteTasks": 8,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 1850, "shuffleWriteBytes":
950}, {"stageId": 343, "numTasks": 2, "numCompleteTasks": 2, "inputBytes":
1086721, "outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 1850},
{"stageId": 342, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 131816,
"outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}
P4_FILES_variant_4 {"seconds": 0.16849635701510124, "jobs": [160, 161], "stages":
[346, 347, 348, 349], "tasks": 15, "shuffle_read": 3534, "shuffle_write": 3534,
"stage_metrics": [{"stageId": 349, "numTasks": 4, "numCompleteTasks": 4,
"inputBytes": 0, "outputBytes": 4268, "shuffleReadBytes": 1684,
"shuffleWriteBytes": 0}, {"stageId": 348, "numTasks": 8, "numCompleteTasks": 8,
"inputBytes": 0, "outputBytes": 0, "shuffleReadBytes": 1850, "shuffleWriteBytes":
1684}, {"stageId": 347, "numTasks": 2, "numCompleteTasks": 2, "inputBytes":
1086721, "outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 1850},
{"stageId": 346, "numTasks": 1, "numCompleteTasks": 1, "inputBytes": 131816,
"outputBytes": 0, "shuffleReadBytes": 0, "shuffleWriteBytes": 0}]}

{
  "name": "B",
  "inputs": {
    "items": {
      "rows": 112650,
      "partitions": 2
    },
    "sellers": {
      "rows": 3095,
      "partitions": 1
    }
  },
  "lazy": {
    "before": [],
    "after_transform": [],
    "after_action": [
      103
    ],
    "rows": 112650
  },
  "join_rows": 112650,
  "smj": {
    "seconds": 0.16632000298704952,
```

```
    "jobs": [
      104
    ],
    "stages": [
      232,
      233,
      234,
      235
    ],
    "tasks": 12,
    "shuffle_read": 1369691,
    "shuffle_write": 1369691,
    "stage_metrics": [
      {
        "stageId": 235,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 0,
        "outputBytes": 0,
        "shuffleReadBytes": 472,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 234,
        "numTasks": 8,
        "numCompleteTasks": 8,
        "inputBytes": 0,
        "outputBytes": 0,
        "shuffleReadBytes": 1369219,
        "shuffleWriteBytes": 472
      },
      {
        "stageId": 233,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 131816,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 106379
      },
      {
        "stageId": 232,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 964388,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 1262840
      }
    ]
  },
  "bhj": {
    "seconds": 0.09229239900014363,
    "jobs": [
      105,
      106
    ],
    "stages": [
      236,
      237,
      238
```

```
  ],
  "tasks": 4,
  "shuffle_read": 118,
  "shuffle_write": 118,
  "stage_metrics": [
    {
      "stageId": 238,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 118,
      "shuffleWriteBytes": 0
    },
    {
      "stageId": 237,
      "numTasks": 2,
      "numCompleteTasks": 2,
      "inputBytes": 964388,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 118
    },
    {
      "stageId": 236,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 131816,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 0
    }
  ]
},
"groupby": {
  "seconds": 0.20214182301424444,
  "jobs": [
    107,
    108,
    109
  ],
  "stages": [
    239,
    240,
    241,
    242,
    243,
    244
  ],
  "tasks": 27,
  "shuffle_read": 5322,
  "shuffle_write": 3472,
  "stage_metrics": [
    {
      "stageId": 244,
      "numTasks": 8,
      "numCompleteTasks": 8,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 1622,
      "shuffleWriteBytes": 0
```

```
    },
    {
      "stageId": 243,
      "numTasks": 8,
      "numCompleteTasks": 8,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 1850,
      "shuffleWriteBytes": 1622
    },
    {
      "stageId": 241,
      "numTasks": 8,
      "numCompleteTasks": 8,
      "inputBytes": 0,
      "outputBytes": 0,
      "shuffleReadBytes": 1850,
      "shuffleWriteBytes": 0
    },
    {
      "stageId": 240,
      "numTasks": 2,
      "numCompleteTasks": 2,
      "inputBytes": 1086721,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 1850
    },
    {
      "stageId": 239,
      "numTasks": 1,
      "numCompleteTasks": 1,
      "inputBytes": 131816,
      "outputBytes": 0,
      "shuffleReadBytes": 0,
      "shuffleWriteBytes": 0
    }
  ]
},
"groups": [
  {
    "seller_state": "SP",
    "lines": 80342,
    "gross": 10235883.88
  },
  {
    "seller_state": "PR",
    "lines": 8671,
    "gross": 1458900.73
  },
  {
    "seller_state": "MG",
    "lines": 8827,
    "gross": 1224159.8
  },
  {
    "seller_state": "RJ",
    "lines": 4818,
    "gross": 937814.12
  },
  {
```

```
  "seller_state": "SC",
  "lines": 4075,
  "gross": 738973.13
},
{
  "seller_state": "RS",
  "lines": 2199,
  "gross": 435802.63
},
{
  "seller_state": "BA",
  "lines": 643,
  "gross": 305262.24
},
{
  "seller_state": "DF",
  "lines": 899,
  "gross": 116243.54
},
{
  "seller_state": "PE",
  "lines": 448,
  "gross": 103886.31
},
{
  "seller_state": "GO",
  "lines": 520,
  "gross": 78964.71
},
{
  "seller_state": "ES",
  "lines": 372,
  "gross": 59860.74
},
{
  "seller_state": "MA",
  "lines": 405,
  "gross": 48550.24
},
{
  "seller_state": "CE",
  "lines": 94,
  "gross": 24600.47
},
{
  "seller_state": "MT",
  "lines": 145,
  "gross": 21702.45
},
{
  "seller_state": "PB",
  "lines": 38,
  "gross": 18584.15
},
{
  "seller_state": "RN",
  "lines": 56,
  "gross": 11296.71
},
{
  "seller_state": "MS",
```

```
      "lines": 50,
      "gross": 9750.65
    },
    {
      "seller_state": "RO",
      "lines": 14,
      "gross": 5474.98
    },
    {
      "seller_state": "PI",
      "lines": 12,
      "gross": 2965.32
    },
    {
      "seller_state": "SE",
      "lines": 10,
      "gross": 1924.69
    },
    {
      "seller_state": "PA",
      "lines": 8,
      "gross": 1393.11
    },
    {
      "seller_state": "AM",
      "lines": 3,
      "gross": 1258.8
    },
    {
      "seller_state": "AC",
      "lines": 1,
      "gross": 299.84
    }
  ],
  "profiles": {
    "repartition": {
      "partitions": 4,
      "sizes": [
        {
          "pid": 0,
          "count": 28162
        },
        {
          "pid": 1,
          "count": 28162
        },
        {
          "pid": 2,
          "count": 28161
        },
        {
          "pid": 3,
          "count": 28165
        }
      ],
      "min": 28161,
      "max": 28165,
      "mean": 28162.5
    },
    "coalesce": {
      "partitions": 4,
```

```
      "sizes": [
        {
          "pid": 0,
          "count": 9721
        },
        {
          "pid": 1,
          "count": 88381
        },
        {
          "pid": 2,
          "count": 9200
        },
        {
          "pid": 3,
          "count": 5348
        }
      ],
      "min": 5348,
      "max": 88381,
      "mean": 28162.5
    }
  },
  "cache": {
    "without": [
      0.17753744797664694,
      0.12260773600428365
    ],
    "materialization": 0.19765729698701762,
    "cached": [
      0.11676288300077431,
      0.163297984021483
    ],
    "checksums": [
      15843553.239999592,
      15843553.240000041,
      15843553.239999592,
      15843553.240000041
    ],
    "storage": [
      {
        "id": 1025,
        "name": "*(2) Project [seller_id#206, price#208, freight_value#209,
gross#2387, seller_zip_code_prefix#236, seller_city#237, seller_state#238]\n+- *
(2) BroadcastHashJoin [seller_id#206], [seller_id#235], Inner, BuildRight, false\n
:- *(2) Project [seller_id#206, price#208, freight_value#209, (price#208 +
freight_value#209) AS gross#2387]\n   :  +- *(2) Filter ((isnotnull(price#208) AND
(price#208 > 0.0)) AND isnotnull(seller_id#206))\n   :     +- *(2) ColumnarToRow\n
:        +- FileScan parquet [seller_id#206,price#208,freight_value#209] Batched:
true, DataFilters: [isnotnull(price#208), (price#208 > 0.0),
isnotnull(seller_id#206)], Format: Parquet, Location: InMemoryFileIndex(1 paths)
[file:/home/albakhtin/tiabd/АлбахтинИВ/Spark/data/spark_input_parquet/o...,
PartitionFilters: [], PushedFilters: [IsNotNull(price), GreaterThan(price,0.0),
IsNotNull(seller_id)], ReadSchema:
struct<seller_id:string,price:double,freight_value:double>\n   +-
BroadcastExchange HashedRelationBroadcastMode(List(input[0, string,
false]),fals...",
        "numPartitions": 2,
        "numCachedPartitions": 2,
        "storageLevel": "Disk Memory Serialized 1x Replicated",
        "memoryUsed": 2778587,
```

```
      "diskUsed": 0,
      "dataDistribution": [
        {
          "address": "10.0.2.15:41997",
          "memoryUsed": 2778587,
          "memoryRemaining": 734759707,
          "diskUsed": 0,
          "onHeapMemoryUsed": 2778587,
          "offHeapMemoryUsed": 0,
          "onHeapMemoryRemaining": 734759707,
          "offHeapMemoryRemaining": 0
        }
      ],
      "partitions": [
        {
          "blockName": "rdd_1025_1",
          "storageLevel": "Memory Serialized 1x Replicated",
          "memoryUsed": 1006060,
          "diskUsed": 0,
          "executors": [
            "driver"
          ]
        },
        {
          "blockName": "rdd_1025_0",
          "storageLevel": "Memory Serialized 1x Replicated",
          "memoryUsed": 1772527,
          "diskUsed": 0,
          "executors": [
            "driver"
          ]
        }
      ]
    }
  ],
  "after_unpersist": []
},
"benchmark": [
  {
    "p": 1,
    "times": [
      0.1470317489875015,
      0.13269337400561199,
      0.11678955401293933
    ],
    "median": 0.13269337400561199
  },
  {
    "p": 2,
    "times": [
      0.1429354000138119,
      0.1387204859929625,
      0.1271599400206469
    ],
    "median": 0.1387204859929625
  },
  {
    "p": 4,
    "times": [
      0.12560606398619711,
      0.12573854500078596,
```

```
      0.14702542099985294
    ],
    "median": 0.12573854500078596
  },
  {
    "p": 8,
    "times": [
      0.14535661999252625,
      0.2189026350097265,
      0.14915432600537315
    ],
    "median": 0.14915432600537315
  }
],
"files": [
  {
    "p": 1,
    "files": 1,
    "total_mib": 0.0013036727905273438,
    "mean_kib": 1.3349609375,
    "metrics": {
      "seconds": 0.15960440601338632,
      "jobs": [
        158,
        159
      ],
      "stages": [
        342,
        343,
        344,
        345
      ],
      "tasks": 12,
      "shuffle_read": 2800,
      "shuffle_write": 2800,
      "stage_metrics": [
        {
          "stageId": 345,
          "numTasks": 1,
          "numCompleteTasks": 1,
          "inputBytes": 0,
          "outputBytes": 1367,
          "shuffleReadBytes": 950,
          "shuffleWriteBytes": 0
        },
        {
          "stageId": 344,
          "numTasks": 8,
          "numCompleteTasks": 8,
          "inputBytes": 0,
          "outputBytes": 0,
          "shuffleReadBytes": 1850,
          "shuffleWriteBytes": 950
        },
        {
          "stageId": 343,
          "numTasks": 2,
          "numCompleteTasks": 2,
          "inputBytes": 1086721,
          "outputBytes": 0,
          "shuffleReadBytes": 0,
```

```
        "shuffleWriteBytes": 1850
      },
      {
        "stageId": 342,
        "numTasks": 1,
        "numCompleteTasks": 1,
        "inputBytes": 131816,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 0
      }
    ]
  }
},
{
  "p": 4,
  "files": 4,
  "total_mib": 0.004070281982421875,
  "mean_kib": 1.0419921875,
  "metrics": {
    "seconds": 0.16849635701510124,
    "jobs": [
      160,
      161
    ],
    "stages": [
      346,
      347,
      348,
      349
    ],
    "tasks": 15,
    "shuffle_read": 3534,
    "shuffle_write": 3534,
    "stage_metrics": [
      {
        "stageId": 349,
        "numTasks": 4,
        "numCompleteTasks": 4,
        "inputBytes": 0,
        "outputBytes": 4268,
        "shuffleReadBytes": 1684,
        "shuffleWriteBytes": 0
      },
      {
        "stageId": 348,
        "numTasks": 8,
        "numCompleteTasks": 8,
        "inputBytes": 0,
        "outputBytes": 0,
        "shuffleReadBytes": 1850,
        "shuffleWriteBytes": 1684
      },
      {
        "stageId": 347,
        "numTasks": 2,
        "numCompleteTasks": 2,
        "inputBytes": 1086721,
        "outputBytes": 0,
        "shuffleReadBytes": 0,
        "shuffleWriteBytes": 1850
```

```
          },
          {
            "stageId": 346,
            "numTasks": 1,
            "numCompleteTasks": 1,
            "inputBytes": 131816,
            "outputBytes": 0,
            "shuffleReadBytes": 0,
            "shuffleWriteBytes": 0
          }
        ]
      }
    }
  ]
}
```


10 Журнал Spark UI и завершение

REST API текущего приложения сохраняет Jobs и Stages. Скриншоты снимаются до spark.stop. Измерения относятся только к этому стенду и текущим данным.

In [11]:

```
save_json("pr4.json",result)
save_json("pr4_jobs.json",ui_data(spark,"/jobs"))
save_json("pr4_stages.json",ui_data(spark,"/stages"))
save_json("pr4_sql.json",ui_data(spark,"/sql?length=1000"))
checkpoint("pr4_completed",spark,["/jobs/","/stages/","/SQL/","/environment/"])
```

```
AUTOMATIC UI CAPTURE pr4_completed 2
```

Выводы и ограничения

При интерпретации сопоставляются медианы, планы и UI. Exchange указывает на перераспределение; Coalesce сохраняет перекос. Broadcast меняет механизм JOIN без изменения строк. Цена persist включает материализацию. Рост p создаёт дополнительные задачи и файлы. AQE способен сократить число выходных shuffle partitions.

Ограничения: два vCPU одной VM не моделируют сеть физического кластера; JVM/JIT, файловый кэш и фоновые службы влияют на короткие замеры. Причинный эффект способа JOIN нельзя выводить только из одного времени запуска.

In [12]:

```
spark.stop()
print("SparkSession stopped")
```

```
SparkSession stopped
```

Приложение Сохранённые снимки Spark UI

Снимки получены во время того же Run All до остановки Spark.

Снимок 1 Окружение Python и Spark

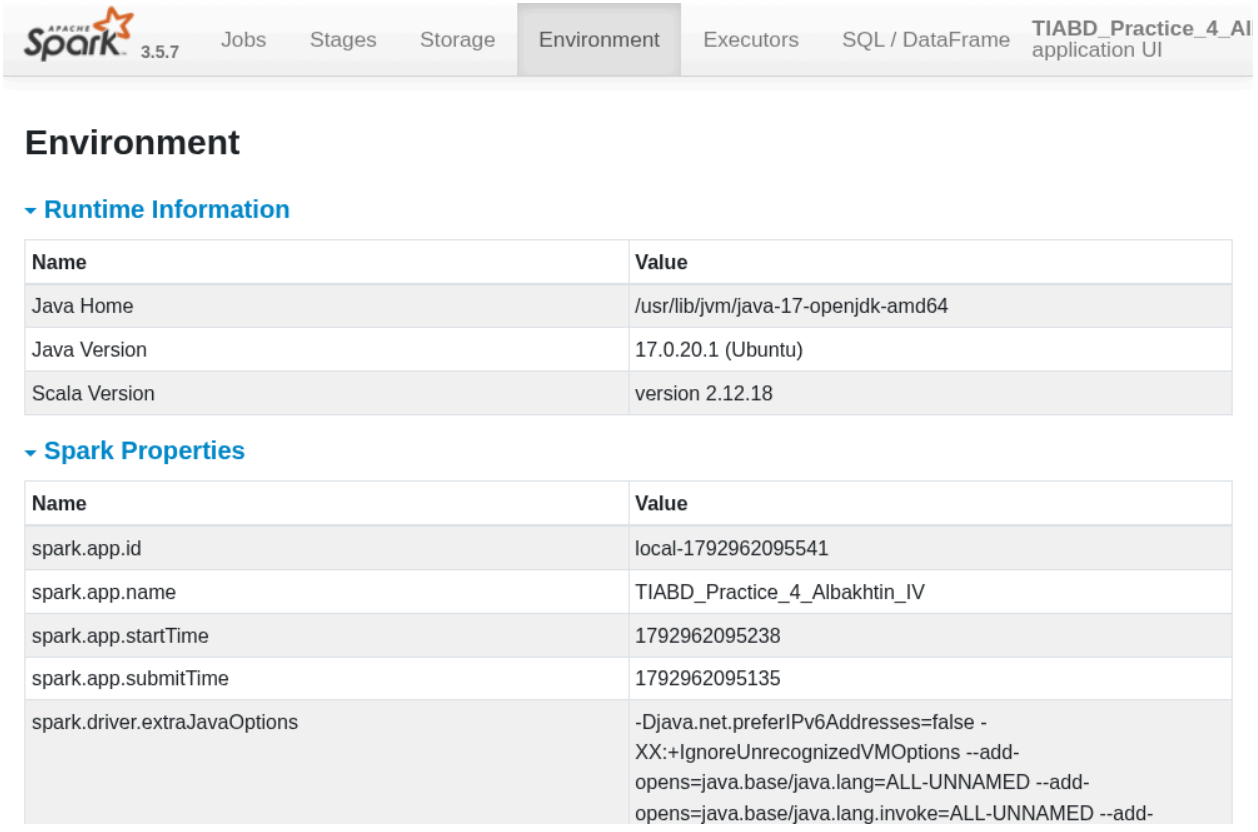


Рисунок 1 – Окружение Python и Spark

Снимок 2 Журнал Jobs основной серии

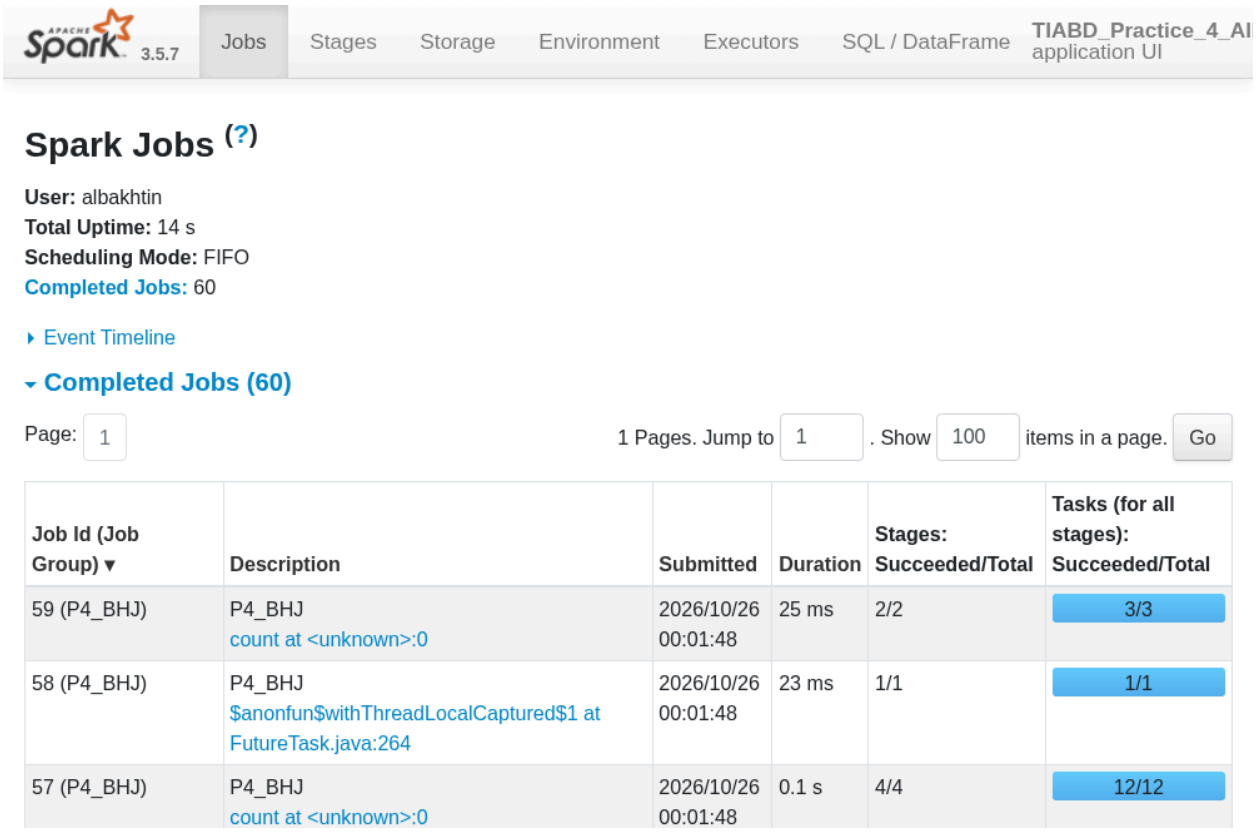


Рисунок 2 – Журнал Jobs основной серии

Снимок 3 Баланс партиций

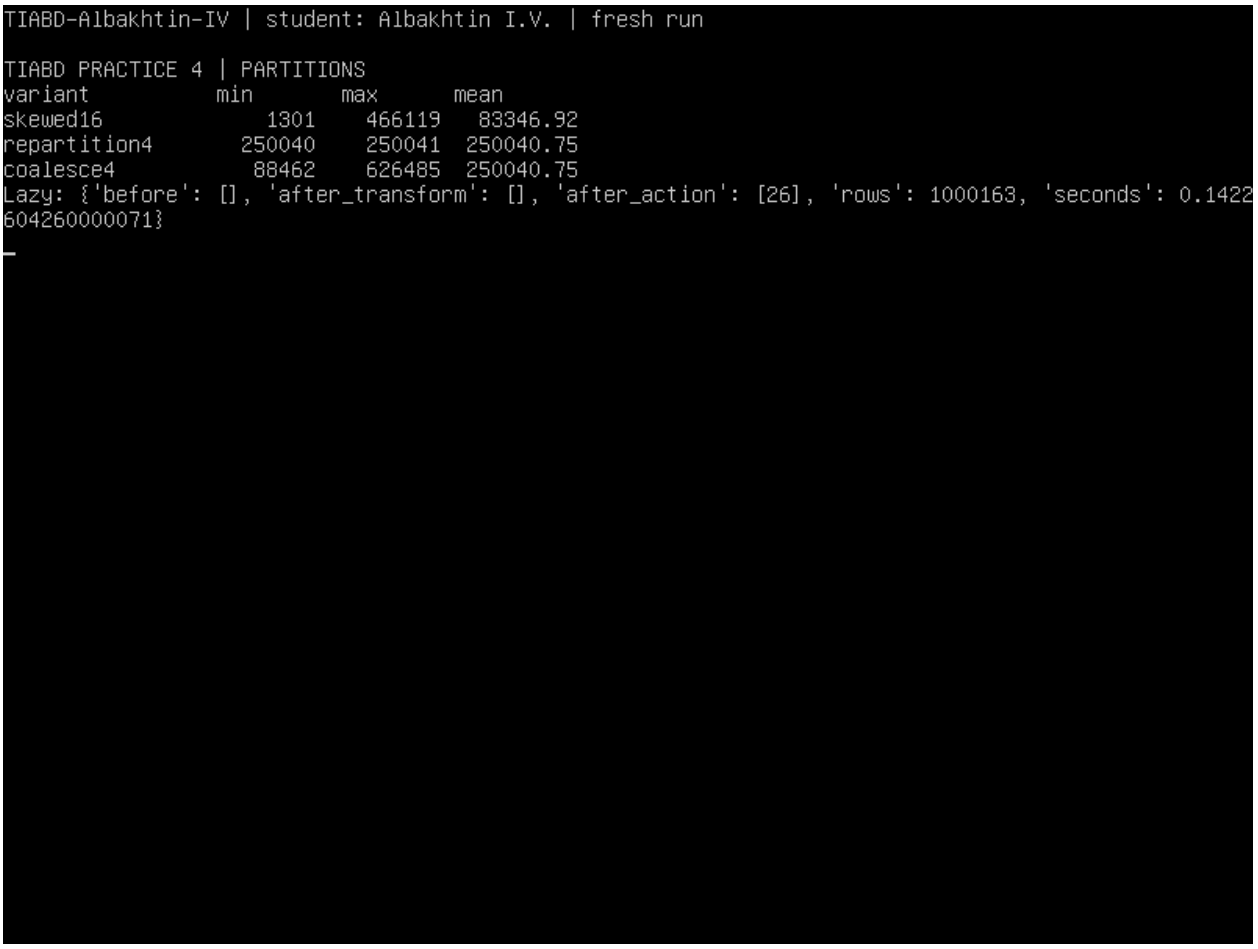


Рисунок 3 – Баланс партиций

Снимок 4 Стадии GroupBy

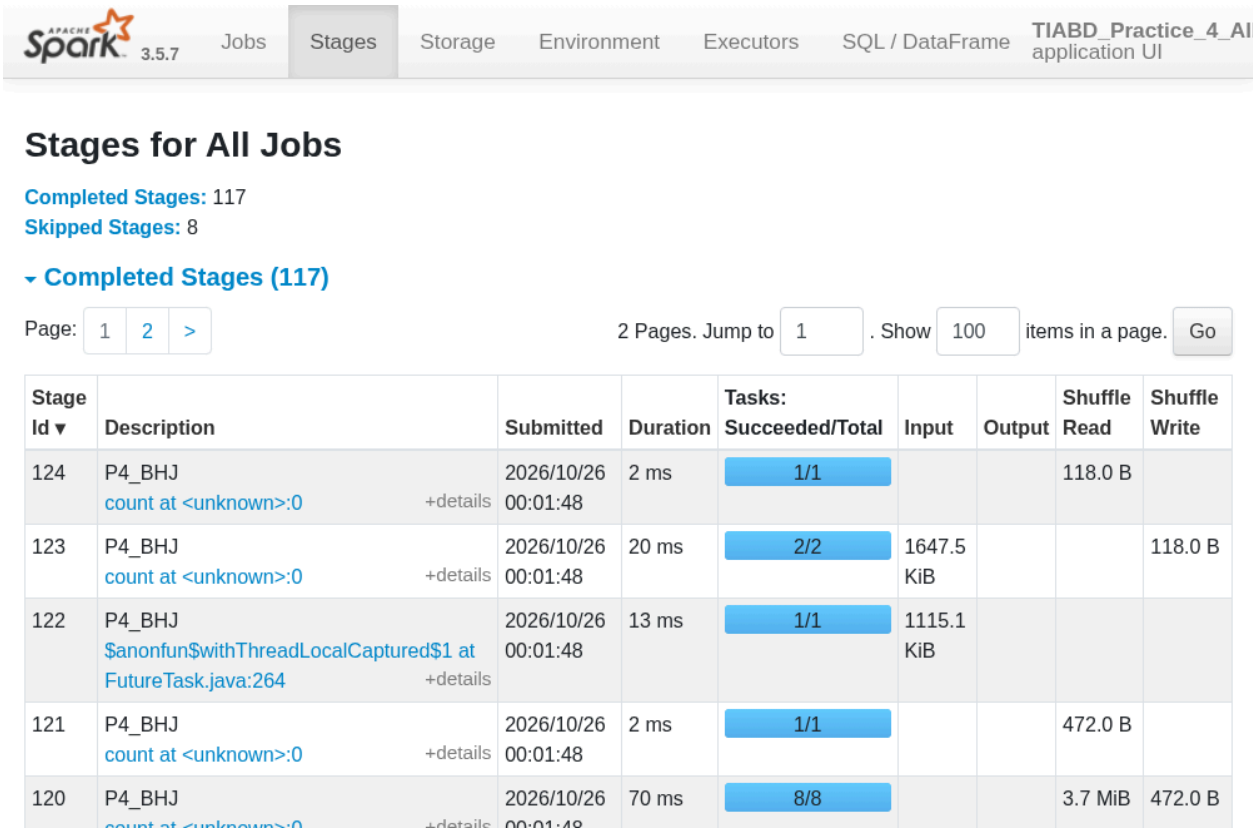


Рисунок 4 – Стадии GroupBy

Снимок 5 Sort Merge Join

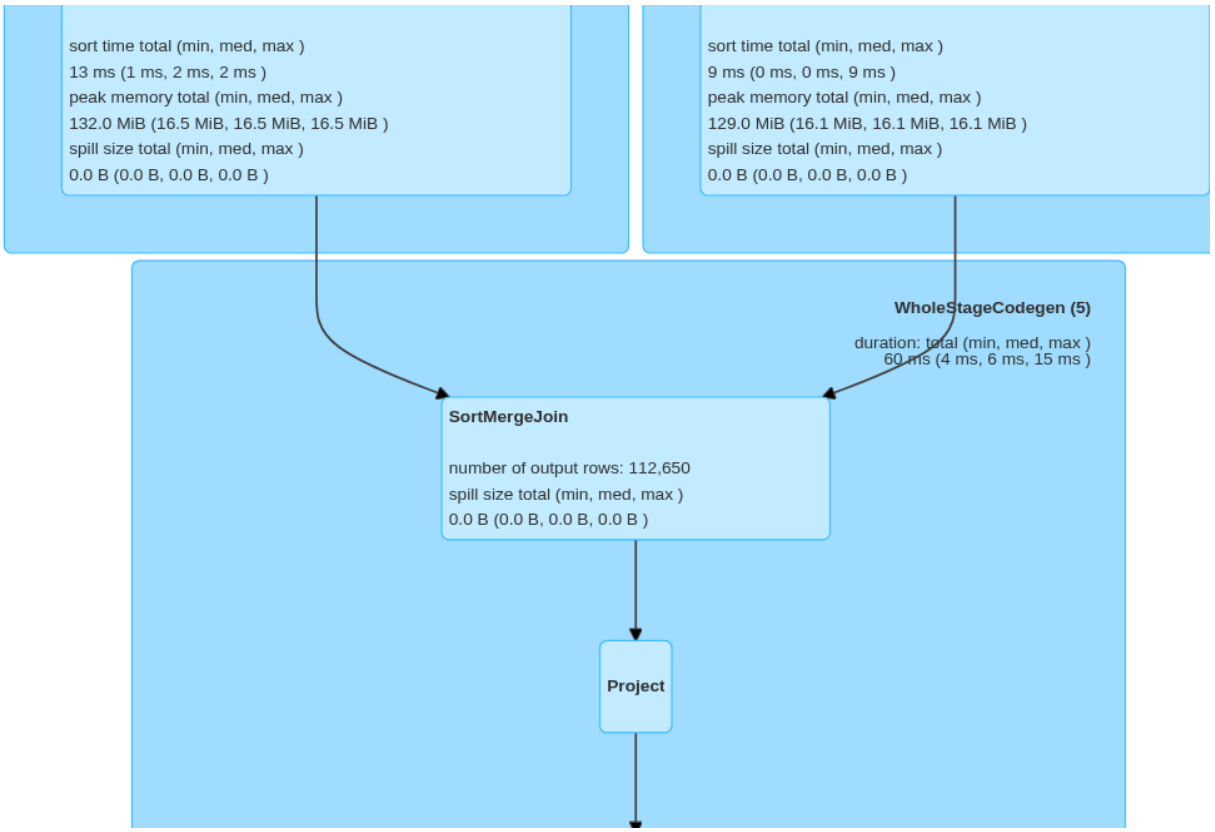


Рисунок 5 – Sort Merge Join

Снимок 6 Broadcast Hash Join

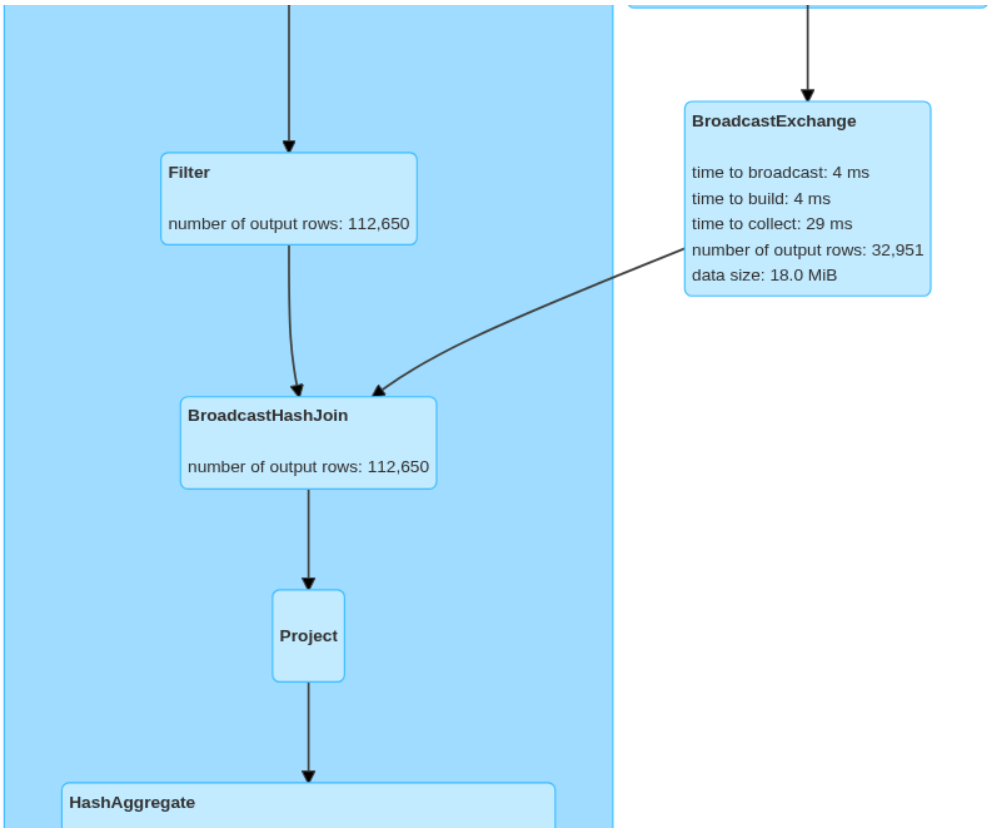


Рисунок 6 – Broadcast Hash Join

Снимок 7 Материализованный persist

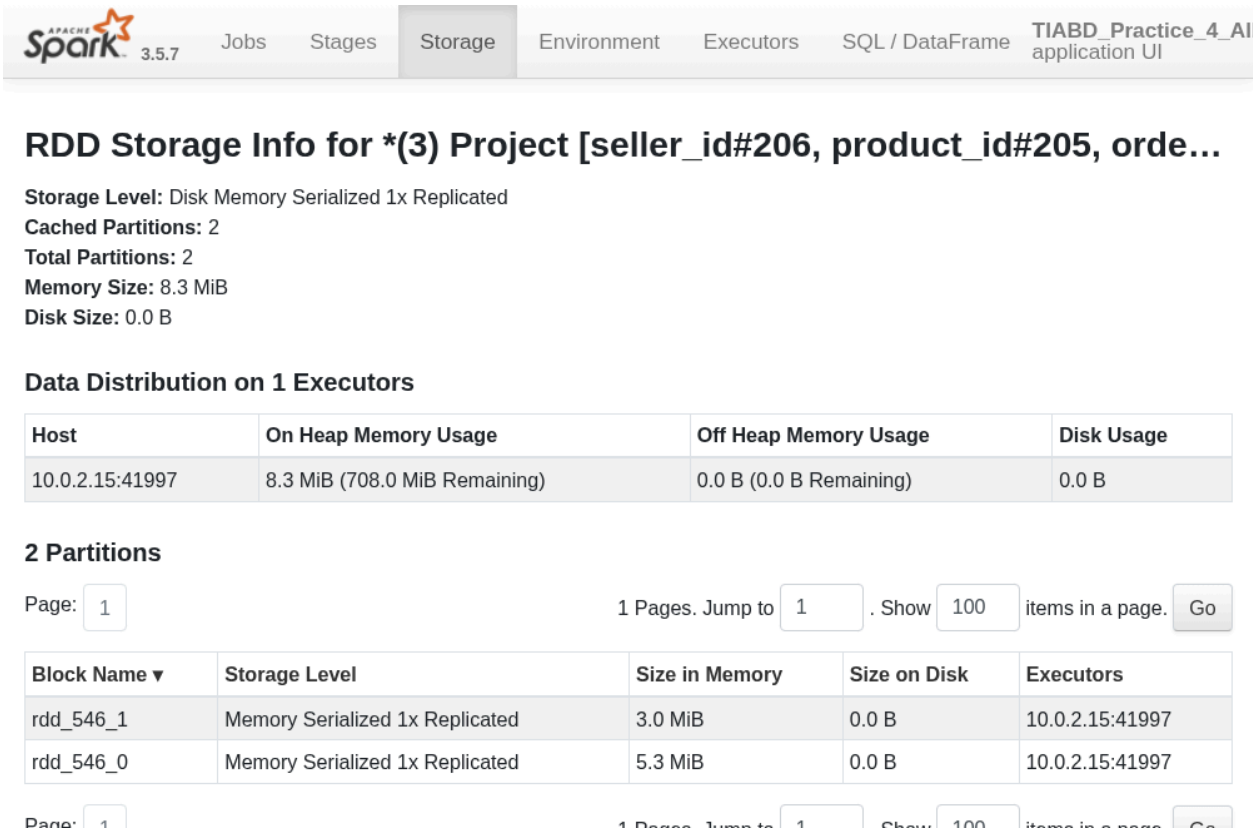


Рисунок 7 – Материализованный persist

Снимок 8 Измерения benchmark

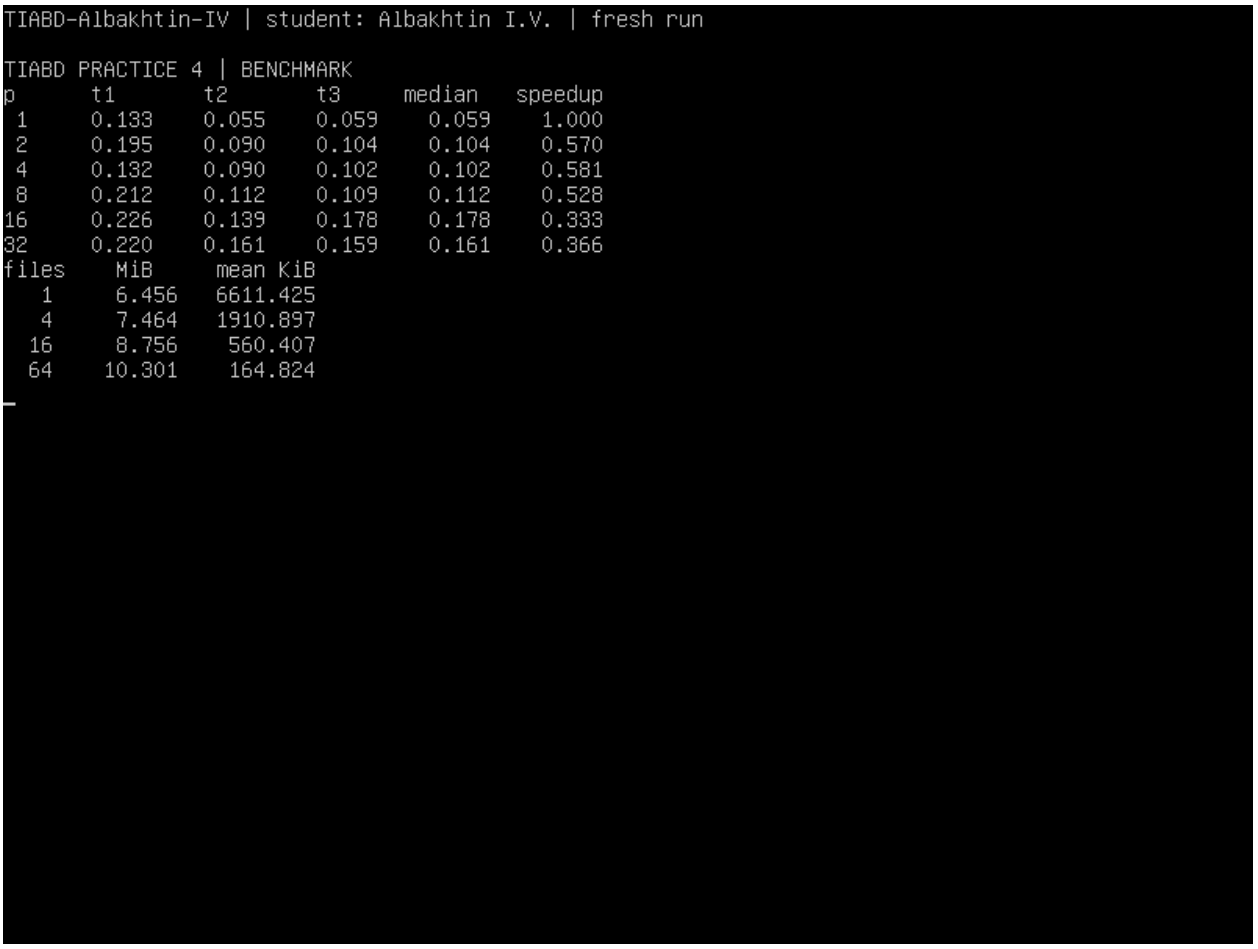


Рисунок 8 – Измерения benchmark

Снимок 9 Persist самостоятельного варианта B

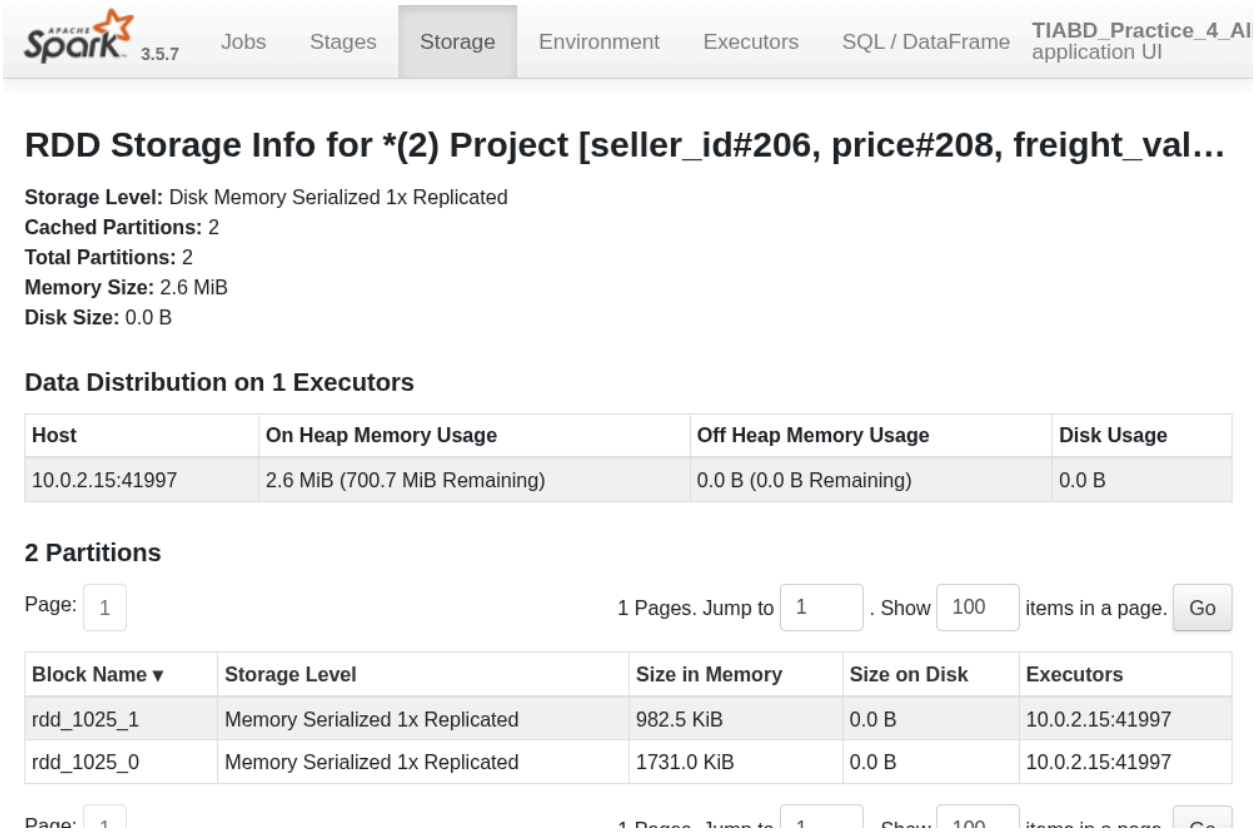


Рисунок 9 – Persist самостоятельного варианта B

Снимок 10 Физический план самостоятельного варианта B

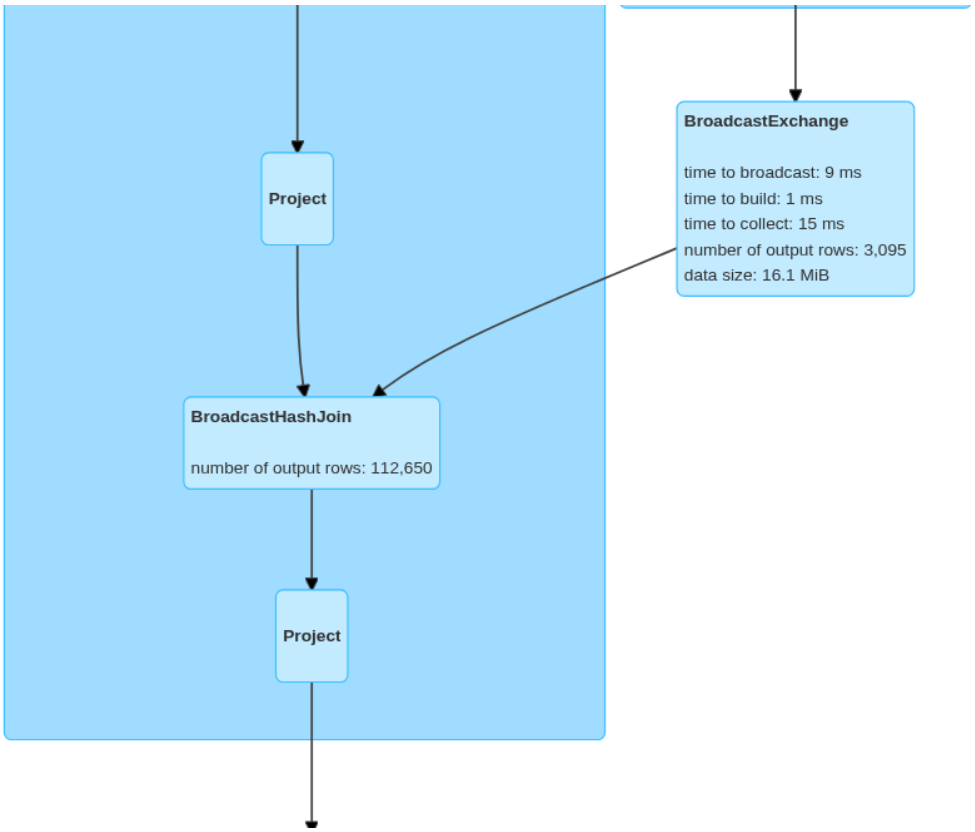


Рисунок 10 – Физический план самостоятельного варианта B

Снимок 11 Стадии самостоятельного варианта B

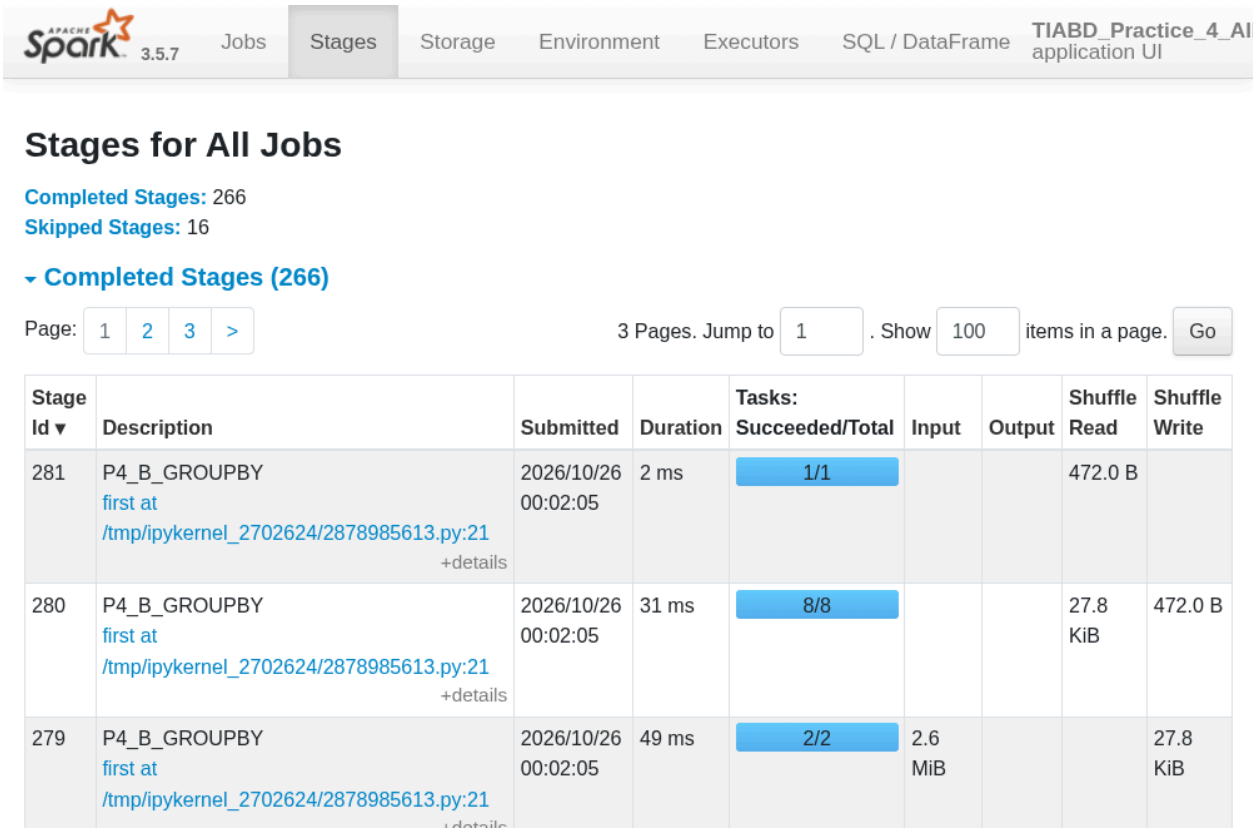


Рисунок 11 – Стадии самостоятельного варианта B

Снимок 12 Завершённые Jobs перед остановкой Spark

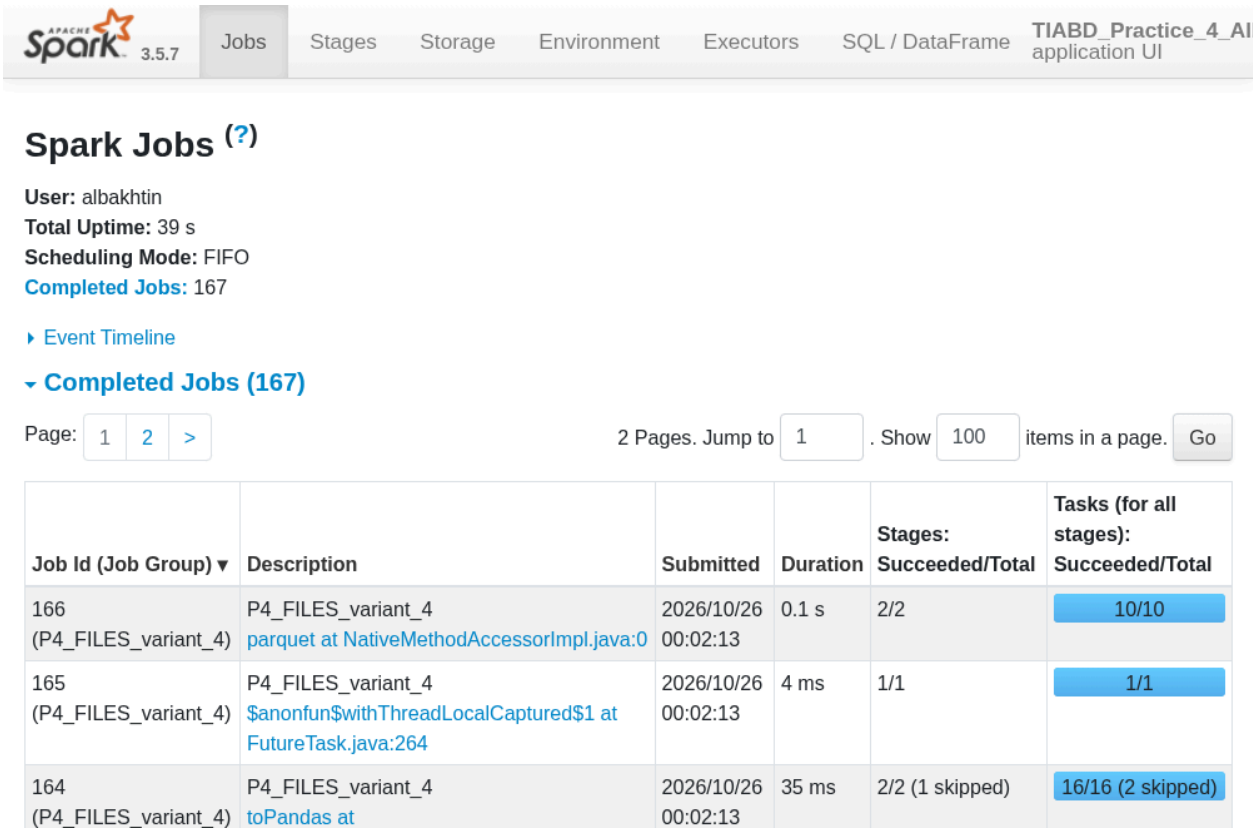


Рисунок 12 – Завершённые Jobs перед остановкой Spark